

object from other objects that communicate with it. Finally, **transaction transparency** allows a system to achieve consistency by masking the coordination among a set of resources. Transactions (see Section 13.10.2, Data Integrity and Log-Structured File Systems) include service requests (such as file access and function calls) that change the state of the system. Consequently, transactions often require checkpointing or replication to meet other goals of the distributed system. Transaction transparency hides the implementation of these services.

Self Review

1. Give an example of location transparency.
2. When failure transparency is implemented by replication, what issue(s) should the designers consider?

Ans: 1) When you access a file in a distributed system, you do not know which server owns the file. 2) The designers should consider consistency among replicated resources.

17.2.5 Network Operating Systems

A **network operating system** accesses resources on remote computers that run independent operating systems, but is not responsible for resource management at remote locations. Consequently, distributed functions are explicit rather than transparent—a user or process must explicitly specify the resource's location to retrieve a networked file or remotely execute an application. The lack of transparency in network operating systems keeps them from providing some of the benefits of distributed operating systems. However, because network operating systems do not need to ensure transparency, they are easier to implement than distributed operating systems.⁴

Self Review

1. State how network operating systems differ from distributed operating systems.
2. Why are network operating systems easier to implement than distributed operating systems?

Ans: 1) The network operating system lacks transparency, which means the client must explicitly specify the resource location to access the resource. 2) Network operating systems do not provide transparency and are not responsible for resource management at remote locations, so network operating systems do not need to implement these capabilities.

17.2.6 Distributed Operating Systems

A **distributed operating system** manages resources located in multiple networked computers. Distributed operating systems employ many of the same communication methods, file system structures and other protocols found in network operating systems, but make the communication transparent so that objects in the system are unaware of the separate computers that provide the service. This level of transparency is difficult to achieve, so it is rare to find a "truly" distributed system.⁵

17.3 Communication in Distributed Systems

One of the primary challenges in designing distributed systems is managing communication between computers. Designers must establish interoperability between heterogeneous computers and applications. In the early years of distributed computing, processes communicated by calling functions from processes located on remote computers. Many of today's applications, however, require interaction between remote objects via method calls. In this section we discuss remote function calls and interactions among remote objects via remote method calls.⁶

Interoperability permits software components to interact among different hardware and software platforms, programming languages and communication protocols. An interface allows heterogeneous systems to communicate in a way that is meaningful to both sides—promoting interoperability. In the client/server model, the client issues a request across the network to the server, and the server processes the request and sends the reply back to the client computer, often using an interface to aid the communication. A **standardized interface** allows each client/server pair to communicate using a single, common interface that is understood by both sides. Standardized interfaces are discussed in Section 17.3.4, CORBA (Common Object Request Broker Architecture).

SelfReview

1. Why is interoperability a challenge in distributed systems?
2. Explain how the client/server communication model works.

Ans: 1) Distributed systems typically consist of heterogeneous computing systems that must somehow be made to interoperate smoothly despite their differences. **2)** In the client/server communication model, the client issues requests across the network to the server, then the server processes the requests and sends the replies back to the client.

17.7.1 Middleware

The machines that compose distributed systems are often heterogeneous—different hardware running different operating systems communicating across different network architectures. In such environments, it is time-consuming and error-prone for the application programmer to provide the routines to convert messages between computers (see the Anecdote, Consequences of Errors). Software known as **middleware** helps provide portability, transparency and interoperability in distributed systems. Portability enables the movement of a system or component from one environment (including both hardware and software) to another without changing the system or component being moved.

Consider a customer purchasing goods from a merchant by writing a check. A bank has issued the customer a book of checks to use for transactions. Upon receipt of a check, the merchant can process it at any bank (even one which did not issue the check), which transparently performs operations to validate the transaction and process the transfer of funds. In this example, the bank (or banks) represents mid-

Middleware and the merchant and customer represent two computers in a distributed system. Similar to the banks, middleware transparently performs operations to validate transactions and initiate network communication. Middleware provides mechanisms to simplify transactions. Instead of visiting the bank to withdraw funds, paying the merchant and having the merchant visit another bank to deposit the funds, the customer can simply write a check which the merchant deposits directly. In a similar manner, middleware facilitates communication and cooperation among the various components of a distributed system by hiding low-level implementation details from users and applications developers.⁷

Middleware provides standard programming interfaces to enable interprocess communication between remote computers. These interfaces provide portability and transparency.⁸ The following sections describe several common implementations and protocols that form the backbone of many distributed systems.

Self Review

1. Why do distributed systems need middleware?
2. How does middleware provide transparency?

Ans: 1) Distributed systems consist of heterogeneous machines. Middleware enables these heterogeneous machines to work together as a single computer. Middleware provides portability, transparency and interoperability. 2) Middleware provides standard interfaces so that heterogeneous computers can communicate with each other. It makes calls to procedures on another computer look like local procedure calls.



Anecdote

Consequences of Errors

For \$125 million, can you answer how many feet are in a meter? On September 23, 1999, engineers lost contact with NASA's Mars Climate Orbiter, a \$125 million satellite orbiting Mars to study its atmosphere. It was later discovered that the Orbiter had entered

the Martian atmosphere, which destroyed the spacecraft. The cause for the error? The project team in Colorado was measuring the trajectory of the spacecraft in standard units (e.g., feet and pounds) but the California team was measuring it in metric units

(e.g., meters and kilograms). The difference between the two measurements caused the teams to interpret the satellite's bearing incorrectly, which eventually led to the orbiter's loss.

Lesson to operating systems designers: Think twice. Question everything. Dig deeper. The consequences of malfunctions in the systems you design could be serious.

17.3.2 Remote Procedure Call (RPC)

In the mid-1970s, the concept of a **remote procedure call (RPC)** was introduced to provide a structured, high-level approach to interprocess communication in distributed systems. A remote procedure call allows a process executing on one computer to invoke a procedure (or function) in a process executing on another computer. This mechanism assumes a client/server model: The client computer issuing the remote procedure call sends its parameters across the network to the server, where the called procedure resides. Execution occurs at the server, and the result (i.e., the return value of the function) is then transmitted across the network to the client.

A goal of RPC was to simplify the process of writing distributed applications by preserving the syntax of a local procedure (or function) call while transparently initiating network communication. In addition to the primary goal of simplicity, RPC also aimed to be efficient and secure.⁹

To provide transparency to programmers of distributed systems, RPC introduces the concept of a stub. A **stub** prepares outbound data for transmission and translates incoming data so that it may be correctly interpreted. To issue an RPC, a client process makes a call (passing the appropriate parameters) to the procedure in the **client stub**. The client stub performs **marshaling of data**, which packages procedure arguments along with the procedure name into a message for transmission over a network.¹⁰

To make the remote procedure call, the client stub passes the message (with marshaled parameters) to the server (Fig. 17.1). Upon receipt of the message from the client stub, the server's operating system transmits the message to the **server stub**. The message is then unmarshaled, and the stub sends the parameters to the appropriate local procedure. When the procedure has completed, the server stub marshals the result and sends it back to the client. Finally, the client stub unmarshals the result, notifies the process and passes it the result. From the standpoint of

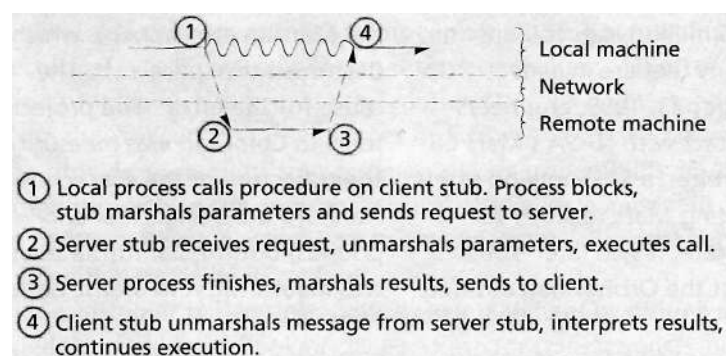


Figure 17.1 | *RPC communication model.*

the client, this looks no different than making a local procedure call and receiving the return result—the elaborate mechanisms of the RPC are hidden.¹¹

There are several complications associated with RPC. RPC can run over either TCP or UDP, which means that different implementations may offer varying levels of reliability. If RPC is operating over UDP, then the RPC implementation must provide communication reliability. Furthermore, each RPC implementation may offer a different level of security (we discuss this topic in Chapter 19, Security and Protection). In fact, we will see that the security capabilities provided by NFS—Section 18.2.2, Network File System (NFS)—are directly related to its underlying RPC implementation.

Another complication is that the process issuing an RPC and its corresponding client stub reside in different memory address spaces. This complicates passing pointers as parameters, which can limit the transparency and capability of RPC. Similarly, RPC does not support global variables, so every variable that a procedure uses must be passed to it as an argument.

Although remote procedure calls promote interoperability in distributed systems, they are not a sufficient solution to the general problem of distributed communication. Performance and security issues have led to the development of additional communication protocols, described in the following sections.

Self Review

1. What is the main benefit of RPC?
2. List several problems with RPC.

Ans: 1) RPC enables processes to call procedures on remote hosts as easily as making calls to procedures on the local machine. 2) Problems with RPCs include: **a)** Lack of reliability and security; **b)** limited transparency and capability; **c)** Does not support global variables; **d)** Complex to implement.

17.3.3 Remote Method Invocation (RMI)

Java's RPC protocol, known as **remote method invocation (RMI)**, enables a Java process executing on one computer to invoke a method of an object on a remote computer using the same syntax as a local method call. Similar to RPC, the details of parameter marshaling and message transport in RMI are transparent to the calling program. A key benefit of RMI is that it allows transmission of objects between remote processes. RMI allows Java programmers to implement distributed systems without having to explicitly program sockets.¹²

Three distinct software layers comprise the RMI architecture: the **stub/skeleton layer**, the **remote reference layer (RRL)** and the **transport layer**.¹³ The stub/skeleton layer contains parameter-marshaling structures analogous to the client and server stubs of RPC. An RMI stub is a Java object residing on the client machine that provides an interface between the client process and the remote object. When a client process invokes a method on a remote object, the stub method is called first. The stub employs **object serialization** to create its marshaled message, a feature

that allows objects to be encoded into byte streams and transmitted from one address space to another. Object serialization enables programs to pass Java objects as parameters and receive objects as return values.¹⁴

Once the parameters have been serialized by the stub, they are sent to the client-side component of the **RMI** system's **RRL**. The **RRL** uses the transport layer to send the marshaled message between the client and the server. When the server-side component of the **RRL** receives the marshaled parameters, it directs them to the **skeleton**, which unmarshals the parameters, identifies the object on which the method is to be invoked and calls that method. Upon completion of the method, the skeleton marshals the result and returns it to the client via the **RRL** and stub.¹⁵

SelfReview

1. What is the function of the stub/skeleton layer in RMI?
2. What benefit does RMI provide over RPC?

Ans: **1)** The stub/skeleton layer marshals and unmarshals parameters. This allows the communication between remote hosts to be hidden from the user. **2)** RMI uses object serialization to enable clients to send objects as arguments in remote method calls and to receive objects as return values from servers.

17.3.4 CORBA (Common Object Request Broker Architecture)

CORBA (Common Object Request Broker Architecture) is a standard specification of distributed systems architecture that has gained wide acceptance.¹⁶ Conceived in the early 1990s by the Object Management Group (OMG), CORBA is an open standard designed to enable interoperation among programs in heterogeneous as well as homogeneous systems. Similar to RMI, CORBA supports objects as parameters or return values in remote procedures during interprocess communication. However, unlike RMI (which is Java based), CORBA is language and system independent, meaning that applications written in different programming languages and on different operating systems interoperate through access to a common CORBA core.¹⁷

CORBA-based distributed systems have a relatively simple structure. The process on the client passes the procedure call along with the required arguments to the client stub. The client stub marshals the parameters and sends the procedure call through its **Object Request Broker (ORB)**, which communicates with the ORB on the server. The ORB on the server then passes off the procedure call to the server's skeleton, which unmarshals the parameters and passes the call to the remote procedure.¹⁸

CORBA provides its users language independence with the **interface definition language (IDL)**. IDL allows programmers to strictly define the procedures that can be called on the object; this is known as that object's interface. Applications written in any language can communicate through CORBA by following each object's IDL specification.¹⁹

Self Review

1. What advantage does CORBA have over RMI?
2. Why might a user choose RMI over CORBA?

Ans: 1) Unlike RMI, CORBA is language independent, so that applications written in different programming languages can interoperate through access to a common CORBA core. 2) If the user is working strictly in Java, then CORBA's language independence is unnecessary. Using CORBA also requires the user to learn IDL so that the client and server can communicate properly.

17.3.5 DCOM (Distributed Component Object Model)

In the 1990s, Microsoft independently developed its own distributed object architecture called **DCOM (Distributed Component Object Model)**.²⁰ 21 DCOM has been included in Windows operating systems since Windows 95 and is CORBA's key competitor in distributed object computing. DCOM is a distributed extension of Microsoft's **Component Object Model (COM)**, which was introduced in 1993 to facilitate component-based development in the Windows environment. The COM specification was designed to allow software components residing in a single address space or in separate address spaces within a single computer to interact with one another.

As in CORBA, objects in DCOM are accessed via interfaces. This allows DCOM objects to be written in a number of different programming languages and on a number of different platforms. Unlike CORBA, however, DCOM objects may have multiple interfaces. When a client requests a DCOM object from a server, the client must also request a specific interface of the object.²² The client request is first sent to the client stub (called a proxy). The client stub communicates over a network with the server stub, which forwards the request to the specific DCOM object. When the DCOM object is finished with the request, it sends the return value back to the server stub. The server stub sends this value back over the network to the proxy, which finally returns the value to the calling process.²³

Self Review

1. How is DCOM similar to CORBA?
2. What is a benefit of having multiple interfaces for a DCOM object?

Ans: 1) Both DCOM and CORBA provide support for remote objects written in different programming languages on different platforms. 2) This allows a single DCOM object to interact differently with different processes.

17.3.6 Process Migration in Distributed Systems

An alternative to remote communication is process migration, which transfers a process between two computers in the distributed system. Recall from Section 15.7 that process migration implements load balancing in multiprocessor systems. Process migration can enable more efficient access to remote resources. For example, consider that computers A and B are networked and that a process residing on computer

A frequently accesses files residing on computer B. The combination of disk and network latency could severely degrade the performance of the process. To reduce this latency, the process from computer A can be transferred to computer B.²⁴

Process cloning creates a copy of a process on a remote machine. Because process cloning does not destroy the original process, the two processes may need to synchronize access to shared memory.²⁵

Self Review

1. Why is process migration employed in distributed systems?
2. Explain the difference between process migration and process cloning.

Ans: 1) Process migration allows a process executing on one computer to be transferred to another to utilize the resources efficiently and to distribute the workload among different processors; it can improve performance by enabling processes to execute without incurring network latencies. 2) In process migration, the process is transferred to, and executed on, a remote computer. In process cloning, a new process is created and executed on the remote computer.

17.4 Synchronization in Distributed Systems

Distributed systems typically contain many processes that cooperate to achieve a common goal. Events at one node in a distributed system often depend on events at other nodes. Determining the order in which events occur is difficult, because communication delays in a distributed network are unpredictable. If two nodes each send a message to a third node, the order of arrival of the two messages might not be the same as the order in which they were sent. This uncertainty can have important consequences if the two messages are related. For example, suppose a distributed system ensures mutual exclusion by having processes broadcast a message whenever they enter or leave a critical section. Suppose one process leaves its critical section and broadcasts a message. Upon receiving this message, another process enters its critical section and broadcasts a second message. If a third process receives the second message, before the first, it will appear to that process that mutual exclusion has been violated.

This mutual exclusion example describes a **causally dependent** relationship. The second message (that a process is executing inside its critical section) may occur only if the first message (indicating that a process has exited its critical section) has been broadcast. **Causal ordering** ensures that all processes recognize that a causally dependent event must occur only after the event on which it is dependent.

Causal ordering is implemented by the **happens-before relation**, which is denoted as $a \rightarrow b$. This relation states that if events a and b belong to the same process, then $a \rightarrow b$ if a occurred before b . This relation also states that if event a is the sending of a message and event b is the receiving of that message, then $a \rightarrow b$. Finally, this relation is transitive, so if $a \rightarrow b$ and $b \rightarrow c$ then $a \rightarrow c$. Causal ordering is only a **partial ordering**, because there will be events for which it cannot be determined which occurred earlier. In this case, these events are said to be **concurrent**.

Sometimes, a causal ordering is not strong enough. A **total ordering** ensures that all events are ordered and that causality is preserved.²⁶

One way to implement total ordering is through a **logical clock** which assigns a number (or **time stamp**) to each event (such as sending and receiving a message, accessing a variable, etc.) that occurs in the system. Logical clocks do not measure time as such, but rather the order in which events occur; they are often implemented as a simple counter of events. Logical clocks enforce a causal ordering in that if $a \rightarrow b$, then the time associated with event a will be less than that associated with event b .²⁷

Scalar logical clocks synchronize the logical clocks on remote hosts and keep a single time value at each host. They ensure causality by following two simple rules. The first rule is that if two events happen within the same process, then the event that happened earlier in actual time will have the earlier time stamp. The second rule is that the event corresponding to receiving a message will have a later time stamp than the event corresponding to sending that message. These two rules also ensure that scalar logical clocks ensure causality.²⁸

To follow the first rule, a process will increment its internal clock each time an event occurs. To follow the second rule, when a process receives a message, it will compare the value of its logical clock to the time stamp from the message, set its logical clock to the latter of the two and then increment it.²⁹

Self Review

1. How is a logical clock implemented?
2. What would happen if the clocks used by different processes were not synchronized?

Ans: 1) To implement the logical clock, a process must increment its clock before it executes an event, and a process must append a time stamp (the local time at which the message was sent) to the end of a message. 2) If the clocks used by different processes were not synchronized, then the time stamp from one process would be interpreted incorrectly by the other process.

17.5 Mutual Exclusion in Distributed Systems

This section discusses how to implement mutual exclusion in distributed systems. The synchronization methods presented in the previous section are used to enforce mutual exclusion.

17.5.1 Mutual Exclusion without Shared Memory

Efficient mutual exclusion algorithms for uniprocessor and shared memory multiprocessors use shared memory. However, in environments with no shared memory, such as NORMA multiprocessors, mutual exclusion must be implemented using some form of message passing, which often requires clock synchronization. **FIFO broadcast** guarantees that when two messages are sent from one process to another, the message that was sent first will arrive first. **Causal broadcast** ensures that when

message M_1 is causally dependent on message M_2 , then no process receives M_1 before receiving M_2 . **Atomic broadcast** guarantees that all messages in a system are received in the same order at each process. Atomic broadcast is also known as **totally ordered broadcast** or **agreed broadcast**.^{30,31}

Self Review

1. What is FIFO broadcast?
2. State the difference between atomic broadcast and causal broadcast.

Ans: 1) FIFO broadcast guarantees that when two messages are sent from one process to another, the message that was sent first will arrive first. 2) Causal broadcast does not guarantee the delivery order of unrelated messages. Atomic broadcast guarantees that unrelated messages are delivered in the same order to each process.

17.5.2 Agrawala and Ricart's Distributed Mutual Exclusion Algorithm

G. Ricart and A. K. Agrawala present an algorithm for ensuring mutual exclusion in NORMA systems in their 1981 paper, "An Optimal Algorithm for Mutual Exclusion in Computer Networks"³² which derives from the work by Lamport.³³ This algorithm requires that a process first send a request message to all other processes in the system and receive a response from each of them before that process can enter its critical section. This algorithm assumes that communication between processes is reliable (i.e., no messages are lost) and that processes do not fail. Algorithms have been developed to address these issues, but they are beyond the scope of this book.

When a process receives a request to enter a critical section and has not sent a request of its own, it sends a reply. If the process has sent its own request, it compares the time stamps of the two requests, and if its own request has a later time stamp than the other request, it sends a reply. If the process's own request has an earlier time stamp than the other request, it delays its reply. Finally, if the time stamps of the requests are equal, the process compares its process number to that of the requesting process. If its own number is higher, it sends a reply, otherwise it delays its reply.

Consider processes P_1 , P_2 , P_3 , each residing on different nodes. Process P_2 wishes to enter its critical section and sends request messages to P_1 and P_3 . Processes P_1 and P_3 do not wish to enter their critical sections and immediately respond. Once P_2 has received both of these replies, it enters its critical section. While P_2 executes inside its critical section, both P_1 and P_3 attempt to enter their critical sections. Each process sends its request message, and both happen to have identical time stamps. Process P_2 receives both requests but delays its reply because it is currently inside its critical section. Process P_3 receives the request from P_1 and compares the time stamps. Because both are equal, P_3 then compares process numbers. P_3 's number is higher than P_1 's, so P_3 sends its reply.

Process P_1 receives the request from P_3 and compares the time stamps. Again, both are equal, but P_1 's number is less than P_3 's, so the reply is delayed. When P_2 exits its critical section, it sends replies to both P_1 and P_3 . Process P_2 does not check either time stamp, since it is concerned only about whether it is currently using its critical

section, not whether another process is doing so. Process P_1 receives the reply from P_2 and enters its critical section (as it has already received a reply from P_3). Process P_3 receives the reply from P_2 but must still wait on the reply from P_1 . Finally, when P_1 is done with its critical section, it sends its reply to P_3 , which will then enter its critical section.

Mutual exclusion is ensured by this algorithm. If two processes entered their critical sections at the same time, it would mean that each process sent a request, received the other process's request, compared the time stamps, and found that the other process's request had a lower time stamp than its own. This is not possible; therefore, mutual exclusion is ensured.³⁴

This algorithm also prevents indefinite postponement. If a process is indefinitely postponed, requests are being sent with earlier time stamps than its own. But because time stamps increase monotonically, all requests sent after the postponed process's request will have a later time stamp. Eventually, all requests with an earlier time stamp will be fulfilled and the postponed process's request will have the lowest timestamp, allowing the postponed process to proceed. In Exercise 17.12 you will be asked to prove that this algorithm prevents deadlock.³⁵

Self Review

1. What are the consequences if a process does fail in this algorithm?
2. If there are n processes within the system, how many messages must be sent before a process can enter its critical section?

Ans: 1) If a process fails, then other processes will not receive replies from it when they send out a request. If there is no mechanism to determine that a process has failed, the entire system could become deadlocked. 2) $2(n - 1)$ messages must be sent.

17.6 Deadlock in Distributed Systems

Distributed deadlock occurs when processes spread over different computers in a network wait for events that will not occur. This section addresses how distributed systems complicate the problem of deadlock and provides algorithms to solve them.

17.6.1 Distributed Deadlock

Distributed deadlock can be classified into three types. Resource deadlock is the type discussed in Chapter 7.³⁶

As in a centralized system, processes in a distributed system also often block, waiting for signals from other processes. This introduces a new type of deadlock called **communication deadlock**, circular waiting for communication signals. For instance, if P_1 is waiting for a response from P_2 , P_2 is waiting for a response from P_3 and P_3 is waiting for a response from P_1 , then the system is deadlocked.³⁷

Due to the communications delay associated with distributed computing, it is possible that a deadlock detection algorithm, introduced in Chapter 7, might detect a deadlock that does not exist. For example, consider processes P_1 and P_2 executing

on different nodes and a third node, N_3 , that is testing for deadlock. Process P_1 holds resource R_1 and process P_2 holds resource R_2 . Process P_1 releases resource R_1 and sends message M_1 to node N_3 . Process P_1 then requests resource R_2 and sends message M_2 to node N_3 . Process P_2 releases resource R_2 and sends message M_3 to node N_3 . Finally, P_2 requests R_1 and sends message M_4 to node N_3 . Suppose that, because of network latency, messages M_2 and M_4 arrive at node N_3 before M_1 and M_3 . Node N_3 would then detect a deadlock which did not exist. This is known as **phantom deadlock**.³⁸

Self Review

1. When would phantom deadlock occur?
2. Why is it difficult to manage distributed deadlock?

Ans: 1) Phantom deadlock occurs when communications between two processes are delayed, which leads to a false global state. 2) In distributed systems, the deadlocked processes are spread over different computers in a network. Managing distributed deadlock requires each process to know the global state of the shared resources and critical sections.

17.6.2 Deadlock Prevention

To prevent deadlocks in uniprocessor machines, it is sufficient to deny one of the four necessary conditions for deadlock (i.e., mutual exclusion, wait for, no preemption and circular wait). These techniques can also be employed to prevent distributed deadlock. Rosenkrantz et al. developed two algorithms designed specifically for distributed systems in their 1978 paper, "System Level Concurrency Control for Distributed Database Systems." Both of these algorithms rely on ordering processes based on when each process was started.³⁹

Wound-Wait Strategy

The **wound-wait deadlock prevention strategy** (Fig. 17.2) breaks deadlock by denying the no-preemption condition. Consider the following situation: there are two processes, P_1 and P_2 , and P_1 was created before P_2 . If process P_1 requests a resource held by P_2 , P_1 **wounds** P_2 . Process P_2 will then be restarted, freeing all of its resources, including the resource requested by P_1 . If P_2 requests a resource held by P_1 , P_2 will wait for P_1 to finish using the resource and release it.⁴⁰

Wait-Die Strategy

The **wait-die deadlock prevention strategy** (Fig. 17.3) prevents deadlock by denying the wait-for condition. Consider the following situation: there are two processes, P_1 and P_2 , and P_1 was created before P_2 . Process P_1 can request a resource held by P_2 and wait for P_2 to release that resource. But if P_2 requests a resource held by P_1 , P_2 **dies**. When P_2 dies, it releases all of its resources and restarts.⁴¹

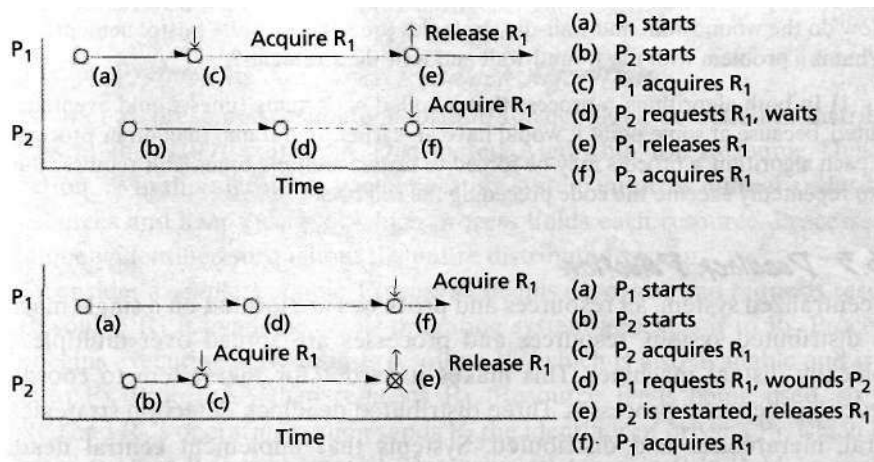


Figure 17.2 | Wound-wait strategy.

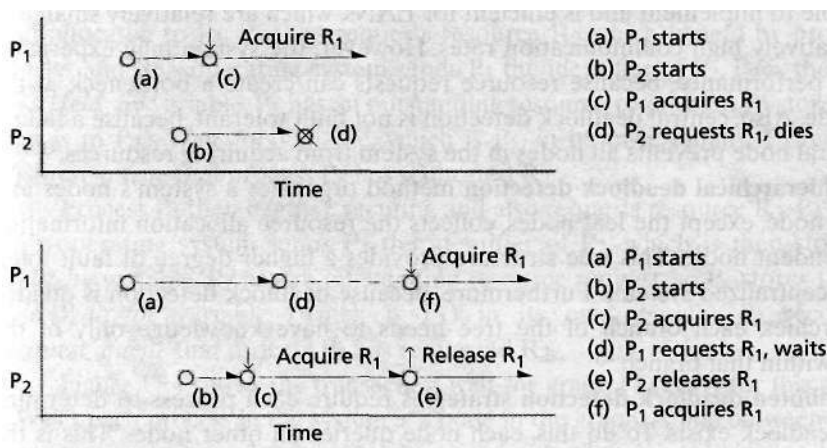


Figure 17.3 | Wait-die strategy.

Both of these strategies prevent deadlock, but processes interact differently in each. In the wound-wait strategy, recently created processes will be restarted if they hold a resource requested by an earlier created process. Recently created processes are also forced to wait for earlier created processes. As a process ages, it will wait less often. In the wait-die strategy, recently created processes will die instead of waiting for a resource. As processes age, they may be forced to wait more often.⁴²

Self Review

1. How do the wound-wait and wait-die strategies prevent indefinite postponement?
2. What is a problem with the wound-wait and wait-die strategies?

Ans: 1) In both algorithms, a process that is rolled back many times would eventually get executed, because at some point it would have an earlier time stamp than other processes.
 2) In each algorithm, a process may be forced to restart multiple times. This requires the process to repeatedly execute the code preceding the roll back.

17.6.3 Deadlock Detection

In a centralized system, all resources and processes are located on a single machine. In a distributed system, resources and processes are spread over multiple, geographically distant machines. This makes it harder for the system to coordinate interactions between processes. Three distributed deadlock detection strategies are central, hierarchical and distributed. Systems that implement **central deadlock detection** contain a single node that monitors the entire system. Whenever a process requests or releases a resource, it notifies the central node. The node checks the global system for cycles. Upon detection of a cycle, the system may implement a variety of deadlock recovery algorithms. The centralized deadlock detection strategy is simple to implement and is efficient for LANs, which are relatively small and exhibit relatively high communication rates. However, the system may experience decreased performance, because resource requests can create a bottleneck at the central node. Also, central deadlock detection is not fault tolerant, because a failure at the central node prevents all nodes in the system from acquiring resources.^{43, 44}

The **hierarchical deadlock detection** method organizes a system's nodes in a tree. Each node, except the leaf nodes, collects the resource allocation information of all dependent nodes. This tree structure provides a higher degree of fault tolerance than centralized systems. Furthermore, because deadlock detection is divided into hierarchies, each branch of the tree needs to have knowledge only of the resources within that branch.⁴⁵

Distributed deadlock detection strategies require each process to determine whether deadlock exists. To do this, each node queries all other nodes. This is the most fault-tolerant method of deadlock detection, because the failure of one node does not prevent other nodes from functioning properly. However, creating efficient algorithms for distributed deadlock detection is difficult, because they must manage synchronized communication among many processes.⁴⁶

Self Review

1. In a distributed system operating over a WAN, why would it be infeasible to use a centralized deadlock detection strategy?
2. What will happen in a hierarchical deadlock detection scheme if a host fails?

Ans: 1) In a WAN, a deadlock may not be detected for a long time, due to communication delays. This will cause processes involved in the deadlock to wait unnecessarily long for the

deadlock to be resolved. 2) Deadlock detection for all hosts dependent on the failed host will stop. This can cause a reduction in performance.

17.6.4 A Distributed Resource Deadlock Algorithm

Johnston et al. presented a simple algorithm for deadlock detection in distributed systems in their 1991 paper, "A Distributed Algorithm for Resource Deadlock Detection."⁴⁷ In this algorithm, each operating system enforces mutual exclusion on its resources and keeps track of which process holds each resource. Processes can be uniquely identified throughout the entire distributed system.

Consider a simple example. Process P_1 begins executing and requests resource R_1 . Resource R_1 is available, so its operating system allocates it to P_1 . Process P_2 then begins executing and requests resource R_2 , which is also available and is allocated to P_2 . Process P_2 then requests R_1 . Resource R_1 is being used, so P_2 is blocked and R_1 's operating system sends P_2 the identifier of process P_1 , which holds the resource that P_2 has requested. This value is stored in P_2 's *Held_by* variable. Process P_1 also holds all of the resources it needs to execute, so P_2 stores P_1 in its *Wait_for* variable. Finally, P_1 records in its *Request_queue* that process P_2 has requested R_1 .

Now process P_3 begins executing. It requests resource R_3 , which is available and allocated to P_3 . P_3 then requests resource R_2 which is held by process P_2 . P_3 blocks, and R_2 's operating system sends P_3 the identifier of P_2 . P_2 is then stored in P_3 's *Held_by* variable. P_2 has an outstanding resource request, so P_3 stores the value that is in P_2 's *Wait_for* variable, which is P_1 , in its own. Finally, P_2 records in its *Request_queue* that process P_3 has requested R_2 .

Process P_4 then begins executing and also requests resource R_2 . P_4 blocks, and R_2 's operating system sends P_4 the identifier of P_2 , which is then stored in P_4 's *Held_by* variable. P_2 has an outstanding resource request, so P_4 stores the value of P_2 's *Wait_for* variable (which is P_1) in its own. Finally, P_2 records in its *Request_queue* that process P_4 has requested R_2 .

Figure 17.4 shows the **transaction wait-for graph (TWFG)** for this system. In a TWFG, each process in the system is a node in the graph. A line between nodes indicates that one process has requested a resource that is being held by another process.

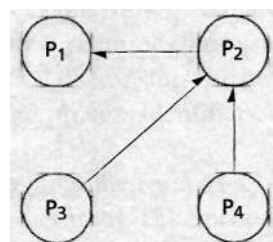


Figure 17.4 | System without deadlocks.

Suppose now that process P_1 requests resource R_3 , which will complete a circular wait (Fig. 17.5). P_1 now blocks, and R_3 's operating system sends P_1 the identifier of the process that holds R_3 , which is P_3 . This process is stored in P_1 's *Held_by* variable. Since P_3 has an outstanding resource request, P_1 stores in its *Wait_for* variable the process in P_3 's *Wait_for* variable, which is P_1 . Finally, P_3 stores in its *Request_queue* that process P_1 has requested R_3 .

When P_1 changes its *Wait_for* variable (from null to P_1), it sends a message to each process in its *Request_queue* to update their own *Wait_for* variables. Process P_2 receives this message, but its *Wait_for* variable is already P_1 , so no change is made. P_2 then checks for deadlock. This is done by testing whether the process in the *Wait_for* variable is also in P_2 's *Request_queue*. The process in P_2 's *Wait_for* variable is P_1 , while the only process in its *Request_queue* is P_3 , so deadlock is not detected at this step. The message is now forwarded to P_3 and P_4 .

Process P_3 receives the message, but its *Wait_for* variable is already P_1 also, so no change is made. P_3 still checks for deadlock, though. The process in P_3 's *Wait_for* variable is P_1 , which is also in P_3 's *Request_queue*, so a deadlock has been detected. In this algorithm, the process that detects the deadlock aborts, so P_3 releases R_3 , notifies P_1 that R_3 has been freed, and aborts. P_1 's resource request will then be filled, and the deadlock has been removed (Fig. 17.6).

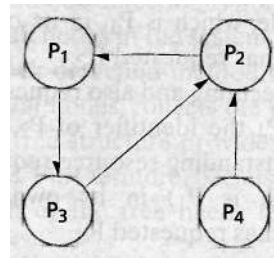


Figure 17.5 | Deadlock is introduced to the system.

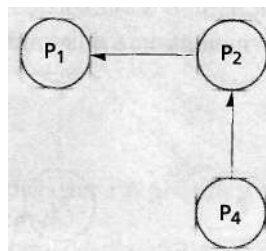


Figure 17.6 | System after deadlock has been eliminated.

SelfReview

1. How does a process check for deadlock in this algorithm?
2. What information is stored in a process's *Request queue*?

Ans: 1) A process checks for deadlock by determining whether the process in its *Wait_for* variable is also an entry in its *Request_queue*. If this is the case, deadlock has occurred. 2) The *Request_queue* keeps track of the processes that have requested a resource held by the original process, and of the resource each of those processes requested.

17.7 Case Study: The Sprite Distributed Operating System

Sprite is a distributed operating system developed at the University of California at Berkeley. Sprite connects large numbers of powerful but inexpensive personal workstations in a high-speed LAN. In a Sprite network, large numbers of personal workstations are connected, and many computers could be idle at any given time. These idle workstations allow Sprite to use process migration to balance the workload of the system.⁴⁸

Each workstation in Sprite monitors its own usage to determine if it is idle. When a workstation does not receive any keyboard or mouse input for 30 seconds and when it has fewer running processes than processors, the workstation claims it is idle and reports to a server, called the **central migration server**, that keeps information about idle workstations. The central migration server will then migrate a process to that workstation, called the **target computer**. When the user of the target computer returns (i.e., the workstation receives a keyboard or mouse input), the workstation notifies the central migration server about the return, and the process is migrated back to its original computer, called the **home computer**.⁴⁹

Each workstation in the system runs a Sprite kernel, and these can communicate with each other via RPCs. Sprite uses **implicit acknowledgements** to ensure that messages are delivered reliably. With implicit acknowledgements, the response message is an implied acknowledgement for the request message.

To ensure transparency, the Sprite kernel distinguishes between two kinds of calls. **Location-dependent calls** are system calls that produce different results for different workstations. **Location-independent calls** are system calls that produce the same result for all workstations. The system then provides more location-independent calls by providing the exact same view of the file system for each workstation. When a location-dependent call is required, the system either forwards the call to the home computer for evaluation or transfers the process's state information (such as virtual memory, open files and process identifiers, etc.) from the home computer to the target computer.

The Sprite file system has the same form as a UNIX file system—every client has the exact same view of the hierarchy. The Sprite file system caches files on both the server and client sides. On the server side, when a file is requested, the server checks the server cache first. If the requested file is not cached, the server adds the file to the server cache and sends it to the client. On the client side, when a file is

requested, the client checks the client cache first. If the file is in the client cache, then the client gets the file from the client cache. Otherwise, the client gets the file from the server and adds it to the client cache.⁵⁰ Section 18.2.5, Sprite File System, discusses the Sprite file system in much more detail.

Self Review

1. Give an example of a location-dependent call.
2. How does Sprite share computer power to balance workload?

Ans: 1) Obtaining the home computer's name is a location-dependent call. 2) Sprite uses a process migration mechanism to enable idle computers to share computer power. When process migration happens, a process on one computer is transferred to an idle computer, distributing the workload among the computers in the system.

17.8 Case Study: The Amoeba Distributed Operating System

Amoeba is a distributed operating system developed at the Vrije Universiteit Amsterdam. Unlike Sprite in which each workstation has its own processors, Amoeba users share processors located in one or more processor pools. A **processor pool** is a collection of processors, each having its own memory and Ethernet connection. The processors in the processor pool are shared equally among users. When a user issues a command to execute a process, the processor pool dynamically allocates the processors for the user. When the user process terminates, the user returns the allocated processors to the processor pool. Like Sprite, Amoeba provides transparency by hiding the number and location of processors from the user.⁵¹

Each processor in an Amoeba system runs a microkernel, which is a collection of services that supports kernel process management, kernel memory management and the communication between clients and servers. It is also the microkernel's responsibility to manage and schedule the threads in a process.⁵²

Amoeba supports two forms of communication—point-to-point and group. In a point-to-point communication, a client stub sends a request message to the server stub and blocks, awaiting the server reply. Amoeba uses **explicit acknowledgement**, which means the client sends an acknowledgement to the server in an additional packet when the client receives the response from the server. In a group communication, messages are sent to all receivers in exactly the same order.

The Amoeba file system has a standard file server called the **bullet** server. The bullet server has a large primary memory. The files stored in it are **immutable**, which means they cannot be modified after being created. If the file is modified, a new file is created to replace the old one, and the old one is deleted from the server.⁵³

The bullet server stores files contiguously on the disk, while Sprite stores files in blocks that are not necessarily contiguous. This means that the bullet server can transfer files faster than Sprite. Unlike Sprite, the bullet server supports only server-side caching. Files located in the cache are also stored contiguously. When a client process wants to access a file, it sends the request to the bullet server. The bullet

server retrieves the file from disk if it is not in the cache, otherwise the bullet server gets the file from the cache.⁵⁴

Self Review

1. How are processors managed in Amoeba?
2. How is a file stored in the bullet server modified?

Ans: 1) All processors are located in one or more processor pools. The processors in a pool are not owned by a particular user but are shared among all the users. The processor pool dynamically allocates the processor for each client. When the processors finish their job to execute a client's command, they are returned to the processor pool. **2)** Bullet servers store only immutable files. To modify a file, a bullet server creates a new file to replace the old one, and the old file is deleted from the server.

Web Resources

www.cs.arizona.edu/computer.help/policy/DIGITAL_unix/AA-Q0R5B-TET1_hmVTOC.html
Discusses the remote procedure call (RPC) model.

www.omg.org/library/wpjava.html
Demonstrates how Java, RMI and CORBA are used in distributed systems.

java.sun.com/products/jdk/rmi/
Java RMI home page, which contains links for white papers, specifications and RMI tutorials.

java.sun.com/j2se/1.4.2/docs/guide/rmi/spec/rmi-TOC.html
Provides the Java platform's remote method invocation system specification.

www.omg.org/gettingstarted/corbafaq.htm
introduces CORBA and provides links to detailed specifications.

www.omg.org/docs/formal/02-12-06.pdf
Detailed CORBA architecture and specification.

msdn.microsoft.com/library/default.asp?url=/library/en-us/dndcom/html/msdn_dcomtec.asp
DCOM technical overview provided by Microsoft.

research.microsoft.com/~ymwang/papers/HTML/DCOMnCORBA/S.html
Architectural comparison of DCOM and CORBA.

<ftp://ftp.cs.vu.nl:/pub/papers/amoeba/comcom91.ps.Z>
Status report on the Amoeba distributed operating system,

www.cs.vu.nl/pub/amoeba/Intro.pdf
Introduction to the Amoeba distributed operating system,

<ftp://ftp.cs.vu.nl:/pub/papers/amoeba/cs91.ps.Z>
Compares the Amoeba and Sprite distributed operating systems.

Summary

In distributed systems, remote computers cooperate via a network to appear as a local machine. Scalability allows the distributed system to grow without affecting the existing applications and users. Distributed systems can be susceptible to attacks by malicious users if they rely on insecure communications media. To improve security, a distributed system should allow only authorized users to access resources and ensure that information transmitted over the network is readable only by the intended recipients. The system must also provide mechanisms to protect resources from attack. Distributed systems implement fault tolerance by providing replication of resources across the system.

Replication offers users increased reliability and availability over single-machine implementations, but designers must provide mechanisms to ensure consistency among the state information at different machines.

Access transparency hides the details of networking protocols that enable communication between distributed computers. Location transparency builds on access transparency to hide the location of resources in the distributed system.

Failure transparency is the method by which a distributed system provides fault tolerance. Under replication, a system provides multiple resources that perform the same

function. A system that employs checkpointing periodically stores the state of an object such that it can be restored if a failure in the distributed system results in the loss of the object.

Replication transparency hides the fact that multiple copies of a resource are available in the system. Persistence transparency hides the information about where the resource is stored—memory or disk.

Migration and relocation transparency both hide the movement of components of a distributed system. The former masks the movement of an object from one location to another in the system; the latter masks the relocation of an object from other objects that communicate with it. Transaction transparency allows a system to achieve consistency by masking the coordination among a set of resources.

A remote procedure call allows a process executing on one computer to invoke a procedure in a process executing on another computer. A goal of RPC was to simplify the process of writing distributed applications by preserving the syntax of a local procedure call while transparently initiating network communication. To issue an RPC, a client process makes a call to the procedure in the client stub, which performs marshaling of data to package procedure arguments along with the procedure name into a message for transmission over a network. The client stub passes the message to the server, which transmits the message to the server stub. The message is then unmarshaled, and the stub sends the parameters to the appropriate local procedure. When the procedure has completed, the server stub marshals the result and sends it back to the client. Finally, the client stub unmarshals the result, notifies the process and passes it the result.

Remote method invocation (RMI) enables a Java process executing on one computer to invoke a method of an object on a remote computer using the same syntax as a local method call. Similar to RPC, the details of parameter marshaling and message transport in RMI are transparent to the calling program.

The stub/skeleton layer of RMI contains parameter-marshaling structures analogous to the client and server stubs of RPC. The stub employs object serialization, which enables programs to pass Java objects as parameters and receive objects as return values. The remote reference layer (RRL) and the transport layer of RMI work together to send the marshaled message between the client and the server. The skeleton unmarshals the parameters, identifies the object on which the method is to be invoked and calls that method. Upon completion of the method, the skeleton

marshals the result and returns it to the client via the RRL and stub.

CORBA (Common Object Request Broker Architecture) is an open standard designed to enable interoperation among programs in heterogeneous as well as homogeneous systems. CORBA supports objects as parameters or return values in remote procedures during inter-process communication and is language independent. In CORBA, the process on the client passes the procedure call along with the required arguments to the client stub. The client stub marshals the parameters and sends the procedure call through its Object Request Broker (ORB), which communicates with the ORB on the server. CORBA provides its users language independence with the Interface Definition Language (IDL), which allows programmers to strictly define the procedures that can be called on the object.

Distributed Component Object Model (DCOM) is designed to allow software components residing on remote computers to interact with one another. As in CORBA, objects in DCOM are accessed via interfaces. Unlike CORBA, however, DCOM objects may have multiple interfaces. When a client requests a DCOM object from a server, the client must also request a specific interface of the object.

Process migration transfers a process between two computers in a distributed system. Process migration between remote computers allows processes to exploit a remote resource, but it is a complicated task that often reduces the performance of the process that is being migrated. Process cloning is similar to process migration, except that instead of transferring a process to a remote location, a new process is created on the remote machine.

Determining the order in which events occur is difficult, because communication delays in a distributed network are unpredictable. Causal ordering ensures that all processes recognize that a causally dependent event must occur only after the event on which it is dependent.

Causal ordering is implemented by the happens-before relation, which states that if events a and b belong to the same process, then $a \rightarrow b$ if a occurred before b . This relation also states that if event a is the sending of a message and event b is the receiving of that message, then $a \rightarrow b$. Finally, this relation is transitive. Causal ordering is only a partial ordering. Events for which it cannot be determined which occurred earlier are said to be concurrent. A total ordering ensures that all events are ordered and that causality is preserved.

One way to implement total ordering is through a logical clock which assigns a timestamp to each event that occurs in the system. Scalar logical clocks synchronize the logical clocks on remote hosts and ensure causality.

In environments with no shared memory, mutual exclusion must be implemented over message passing. Message-passing systems use clock synchronization concepts to employ either FIFO broadcast, causal broadcast or atomic broadcast to synchronize the system. FIFO broadcast guarantees that when two messages are sent from one process to another, the message that was sent first will arrive first. Causal broadcast ensures that when message m_1 is causally dependent on message m_2 , then no process delivers m_1 before delivering m_2 . Atomic broadcast guarantees that all messages in a system are received in the same order at each process.

Ricart and Agrawala's algorithm requires that a process first send a request message to all other processes in the system and receive a response from each of these processes before that process can enter its critical section. When a process receives a request to enter a critical section and has not sent a request of its own, it sends a reply. If the process has sent its own request, it compares the timestamps of the two requests and if the process's own request has a later timestamp than the other request, it sends a reply. If the process's own request has an earlier timestamp than the other request, it delays its reply. Finally, if the timestamps of the requests are equal, the process compares its process number to that of the requesting process. If its own number is higher, it sends a response, otherwise it delays its response.

There are three types of distributed deadlock: resource deadlock, communication deadlock and phantom deadlock. Resource deadlock is the kind of deadlock introduced in Chapter 7. Communication deadlock is circular waiting for communication signals. Due to the communications delay associated with distributed computing, it is possible that a deadlock detection algorithm might detect a deadlock (called phantom deadlock, a perceived deadlock) that does not exist. Although this form of deadlock cannot immediately stop the system, it is a source of inefficiency.

Two algorithms designed to prevent deadlock rely on ordering processes based on when each process was started. The wound-wait strategy breaks deadlock by denying the no-preemption condition. A process will wait for another process if the first process was created before the other. In this strategy, a process will wound (restart) another process if the first process was created after the other. The wait-die strategy breaks deadlock by denying the wait-for condition. In this strategy, a process will wait

for another process if the first process was created after the other process. A process will die (restart) itself if it was created before the other process.

Systems that implement central deadlock detection have one site that is dedicated to monitoring the entire system. Whenever a process requests or releases a resource it informs the central site which continuously checks the system for cycles. The central algorithm is simple to implement and is efficient for LANs. However, the system may experience decreased performance and is not fault tolerant.

The hierarchical deadlock detection method arranges each site in the system as a node in a tree. Each node, except the leaf nodes, collects the resource allocation information of all dependent nodes. This tree structure helps to ensure fault tolerance. Furthermore, because deadlock detection is divided into hierarchies and clusters, sites that are irrelevant to deadlock detection on a particular resource do not have to participate in deadlock detection.

Distributed deadlock detection algorithms place the responsibility of deadlock detection with each site. Each site in the system queries all other sites to determine whether any other sites are involved in deadlock. This is the most fault-tolerant method of deadlock detection because the failure of one site will not cause any other site to fail.

Johnston, et al. presented a simple algorithm for deadlock detection in distributed systems. In this algorithm, each process keeps track of the transaction wait-for graph (TWFG) of which they are involved. When a process requests a resource that is being held by another process, the requesting process blocks and the TWFG is updated. As this happens, any deadlocks are detected and removed.

In a Sprite network, large numbers of personal workstations are connected and many computers could be idle at any given time. These idle workstations allow Sprite to use process migration to balance the workload of the system. When the central migration server is notified that a workstation is idle, it will migrate a process to that target computer. When the user of the target computer returns, the workstation notifies the central migration server about the return, and the process is migrated back to the home computer.

The Sprite kernel provides more location-independent calls by providing the exact same view of the file system for each workstation. When a location-dependent call is required, the system either forwards the call to the home computer for evaluation or transfers the process's state information from the home computer to the target computer. The Sprite file system also caches files on both the server and client sides.

Amoeba users share processors located in one or more processor pools. When a user issues a command to execute a process, the processor pool dynamically allocates the processors for the user. When the user process terminates, the user returns the allocated processors to the processor pool. Amoeba provides transparency by hiding the number and location of processors from the user.

Amoeba supports two forms of communication—point-to-point and group. In a point-to-point communication, a client stub sends a request message to the server

stub and blocks, awaiting the server reply. In a group communication, messages are sent to all receivers in exactly the same order.

The Amoeba file system has a standard file server called the bullet server which has a large primary memory. The files stored in the bullet server are immutable. If a file is modified, a new file is created to replace the old one, and the old one is deleted from the server. The bullet server also stores files contiguously on the disk so that it can transfer files faster than Sprite.

Key Terms

access transparency—Hides the details of networking protocols that enable communication between computers in a distributed system.

atomic broadcast—Guarantees that all messages in a system are received in the same order at each process. Also known as totally ordered or agreed broadcast.

bullet server—Standard file server used in the Amoeba file system.

causal broadcast—Ensures that when a message is sent from one process to all other processes, any given process will receive the message before it receives a response to the message from a different process.

causal ordering—Ensures that all processes recognize that a causally dependent event must occur only after the event on which it is dependent.

causally dependent—Event B is causally dependent on event A if event B may occur only if event A occurs.

central deadlock detection—A strategy in distributed deadlock detection, in which one site is dedicated to monitoring the TWFG of the entire system. Whenever a process requests or releases a resource, it informs the central site. The site continuously checks the global TWFG for cycles.

central migration server—Workstation in a Sprite distributed operating system that keeps information about idle workstations.

client stub—Stub at the client side that prepares outbound data for transmission and translates incoming data so that it may be correctly interpreted.

Component Object Model (COM)—Introduced in 1993 by Microsoft to facilitate component-based development in the Windows environment.

Common Object Request Broker Architecture (CORBA)—Conceived in the early 1990s by the Object Management

Group (OMG), CORBA is a standard specification of distributed systems architecture that has gained wide acceptance.

communication deadlock—One of the two types of distributed deadlock, which is a circular waiting for communication signals.

concurrent—Two events are concurrent if it cannot be determined which occurred earlier by following the happens-before relation.

Distributed Component Object Model (DCOM)—Distributed systems extension of Microsoft's COM.

distributed deadlock—Similar to deadlock in a uniprocessor system, except that the processes concerned are spread over different computers.

distributed deadlock detection strategy—Technique used to find deadlock in a distributed system.

distributed file system—Places files on physically distinct machines, while providing the view of a single file system.

distributed operating system—Provides all of the same services as a traditional operating system, but must provide adequate transparency such that objects in the system are unaware of the computers that provide the service.

distributed system—Remote computers cooperate via a network to provide a unified operating system.

explicit acknowledgement—The client sends an acknowledgement to the server in an additional packet when the client receives the response from the server.

failure transparency—Method by which a distributed system provides fault tolerance so that the client is unaware of the failure of a computer.

FIFO broadcast—Guarantees that when two messages are sent from one process to another, the message that was sent first will arrive first.

happens before relation—A happens before B if A and B belong to the same process and A occurred before B; or A is the sending of a message and B is the receiver of that message.

hierarchical deadlock detection—A strategy in distributed deadlock, which arranges each site in the system as a node in a tree. Each node, except the leaf nodes, collects the resource allocation information of all dependent nodes.

home computer (in process migration)—Computer on which the process originates.

implicit acknowledgement—The response message implies the acknowledgement for the request message.

immutable file—A file that cannot be modified after it is created.

Interface Definition Language (IDL)—A language used to specify the details of the RPCs, which provides a language-independent representation of interfaces and allows distributed applications to transparently call procedures on remote computers.

interoperability—Permits software components to interact among different hardware and software platforms, programming languages and communication protocols.

location-dependent call—System call that depends on the workstation (in Sprite) on which the call is executed. Location-dependent calls produce different results for different workstations.

location-independent call—System call that does not depend on the workstation on which the call is executed. Location-independent calls produce the same result for all workstations.

location transparency—Builds on access transparency to hide the location of resources in a distributed system from those attempting to access them.

logical clock—Assigns a timestamp to each event that happens in a system in order to create a total ordering of events.

marshaling of data—A routine for the client stub to package procedure arguments and return values for transmission over a network.

middleware—A software layer that helps provide portability, transparency, and interoperability in distributed systems.

migration transparency—Masks the movement of an object from one location to another in the system, such as the movement of a file from one server to another.

network operating system—Accesses resources on remote computers that run independent operating systems.

Object Request Broker (ORB)—Component residing on both CORBA client and server, which is responsible for initiating communication between systems.

object serialization—Allows objects to be encoded into byte streams and transmitted from one address space to another.

partial ordering—An ordering of events that follows the happens-before relation. Some events cannot be ordered using this system which is why it is only a partial ordering.

persistence transparency—Hides the information about where the resource is stored—memory or disk.

phantom deadlock—Situation due to communications delay associated with distributed computing, when a deadlock detection algorithm (DDA) might detect a deadlock that does not exist.

process cloning—Creates a copy of a process on a remote machine.

processor pool—Component in Amoeba system, which contains a collection of processors, each having its own memory and Ethernet connection.

proxy—In DCOM, the client-side stub that is responsible for marshaling and unmarshaling messages.

relocation transparency—Masks the relocation of an object from other objects that communicate with it.

remote method invocation (RMI)—Allows Java programmers to implement distributed systems without having to explicitly program sockets.

remote procedure call (RPC)—Allows a process executing on one computer to invoke a procedure (or function) in a process executing on another computer.

remote reference layer (RRL)—Works with the transport layer to send marshaled messages between the client and server in RMI.

replication—Provides multiple resources that perform the same function in a system.

replication transparency—Hides the fact that multiple copies of a resource are available in the system. All access to a group of replicated resources occurs as if there were one such resource available.

scalability—Allows a distributed system to grow (i.e., add more machines to the system) without affecting the existing applications and users.

server stub—A stub at the server side in RPC that prepares outbound data for transmission and translates incoming data so that it may be correctly interpreted.

skeleton—Server-side stub.

810 Introduction to Distributed Systems

standardized interface—Allows each client/server pair to communicate using a single, common interface that is understood by both sides.

state information—Data that describes the status of one or more resources.

stub—prepares outbound data for transmission and translates incoming data so that it may be correctly interpreted

stub/skeleton layer in RMI—Contains parameter-marshaling structures analogous to the client and server stubs of RPC.

target computer (in process migration)—Computer to which the process is migrated.

time stamp—Records the local time at which the message was sent.

total ordering—Ensures that all events are observed in the same order by all processes.

transaction transparency—Allows a system to achieve consistency by masking the coordination among a set of resources.

transaction wait-for graph (TWFG)—Graph that represents processes as nodes and dependencies as directed edges, which is used for distributed deadlock detection algorithms.

transparency—Hides the distribution aspects from users of a distributed system.

transport layer (in RMI)—Works with the RRL to send marshaled messages between the client and server in RMI.

wait-die deadlock prevention strategy—Prevents deadlock by denying the wait-for condition. Assigns individual processes unique priorities based on when they were created. A process will wait if it was created after the process it is waiting on. A process will die if it was created before the process it is waiting on.

wound-wait deadlock prevention strategy—Prevents deadlock by denying the no-preemption condition. Assigns individual processes unique priorities based on when they were created. A process requesting a resource held by another process will wound that process if the first one was created before the other. A process will wait if it was created after the process it is waiting on.

wound—When a process is wounded by another process, it will be rolled back.

Exercises

17.1 What are the benefits of distributed systems?

17.2 Define each type of transparency.

17.3 What is the role of middleware? Give examples of middleware.

17.4 Explain how RPC works.

17.5 When the client and the server communicate via RPC, the client blocks until it receives a response from the server. What is the drawback of this design? Give an alternative suggestion to remove this limitation.

17.6 Why is the Interface Definition Language (IDL) important in CORBA?

17.7 Discuss the major differences in DCOM and CORBA.

17.8 Give examples in which process migration is used to exploit resource locality and to balance work load.

17.9 Draw two diagrams, one illustrating that event e_2 is dependent on e_1 , the other illustrating that events e_2 and e_1 are concurrent.

17.10 Distinguish FIFO broadcast, causal broadcast and atomic broadcast.

17.11 Suppose processes P_1 , P_2 and P_3 share the critical section. P_1 sends a request to enter the critical section at logical time 1. P_2 sends a request to enter the critical section at logical time 2. P_3 sends a request to enter the critical section at logical time 3. Apply Ricart and Agrawala's algorithm to demonstrate how mutual exclusion is achieved.

17.12 Prove that Ricart and Agrawala's algorithm prevents deadlock.

17.13 Suppose processes P_1 , P_2 and P_3 have time stamps 1.2 and 3, respectively. If P_1 requests a resource held by P_2 , which process will be rolled back using the wound-wait algorithm?

17.14 Suppose processes P_1 , P_2 and P_3 have timestamps 1.2 and 3, respectively. If P_3 requests a resource held by P_2 , which process will be rolled back using the Wait-Die algorithm?

17.15 Compare the Sprite and Amoeba distributed systems.

Recommended Reading

This chapter has only provided a high-level overview of certain middleware technologies. More information on CORBA can

be found through the Object Management Group (OMG).⁵⁵ In addition, many of the algorithms introduced in this chapter

are discussed in much greater detail in the papers that introduced each algorithm. Agrawala and Ricart's paper provides much greater detail about their mutual exclusion algorithm.⁵⁶ Rosenkrantz et. al. discuss in their paper concurrency control

in distributed systems.⁵⁷ Other resources are the paper by Johnston et al.,⁵⁸ Which describes their deadlock detection algorithm, and Obermarck's paper,⁵⁹ which covers another deadlock detection algorithm not mentioned in this chapter.

Works Cited

1. Tanenbaum, A. S., and M. van Steen, *Distributed Systems, Principles and Paradigms*, Upper Saddle River, NJ: Prentice Hall, 2002, pp. 4-5.
2. Gartner, F., "Fundamentals of Fault-Tolerant Distributed Computing in Asynchronous Environments," *ACM Computing Surveys*, Vol. 31, No. 1, March 1999.
3. ISO "Open Distributed Processing—Reference Model: Architecture," *Ref. Number: ISO/IEC 10746-3:1996E*, September 15, 1996, pp. 45-48.
4. Renesse, R. V., and A. S. Tanenbaum, "Distributed Operating Systems," *Computing Surveys*, Vol. 17, No. 4, December 1985, pp. 421-422.
5. Renesse, R. V., and A. S. Tanenbaum, "Distributed Operating Systems," *ACM Computing Surveys*, Vol. 17, No. 4, December 1985, pp. 420 and 425.
6. Wegner, P., "Interoperability," *ACM Computing Surveys*, Vol. 28, No. 1, March 1996, pp. 287.
7. Emmerich, W., "Software Engineering and Middleware: A Roadmap," *Proceedings of the Conference on the Future of Software Engineering 2000*, pp. 117-129.
8. Bernstein, P., "Middleware: A Model for Distributed System Services," *Communications of the ACM*, Vol. 39, No. 2, February 1996, pp. 86-98.
9. Birrell, A., and B. Nelson, "Implementing Remote Procedure Calls," *ACM Transactions on Computer Systems*, Vol. 2, No. 1, February 1984, pp. 41-42.
10. Birrell, A., and B. Nelson, "Implementing Remote Procedure Calls," *ACM Transactions on Computer Systems*, Vol. 2, No. 1, February 1984, pp. 43-44.
11. Birrell, A., and B. Nelson, "Implementing Remote Procedure Calls," *ACM Transactions on Computer Systems*, Vol. 2, No. 1, February 1984, pp. 43-44.
12. "Java™ Remote Method Invocation (RMI)," <java.sun.com/products/jdk/rmi/>.
13. "Java™ Remote Method Invocation," <java.sun.com/market-ing/collateral/rmi_ds.html>.
14. "Java™ Remote Method Invocation: Stubs and Skeletons," <java.sun.com/j2se/1.4/docs/guide/rmi/spec/rmi-arch2.html>.
15. "Java™ Remote Method Invocation: 2—Distributed Object Model," 2001, Sun Microsystems, <java.sun.com/j2se/1.4.1/docs/guide/rmi/spec/rmi-objmodel2.html>.
16. "CORBA Basics," <www.omg.org/gettingstarted/corbafaq.htm>.
17. "The Common Object Request Broker: Architecture and Specification," *OMG Document Number 93.12.43*, December 1993.
18. Chung, P., et al., "DCOM and CORBA Side by Side, Step by Step, and Layer by Layer," *C++ Report*, Vol. 10, No. 1, January 1998, pp. 18-29.
19. "The Common Object Request Broker: Architecture and Specification," *OMG Document Number 93.12.43*, December 1993.
20. "DCOM Technical Overview," <msdn.microsoft.com/library/default.asp?url=/library/en-us/dndcom/html/msdn_dcomtec.asp>.
21. Botton, D., "Interfacing ADA 95 to Microsoft COM and DCOM Technologies," *ACM SIGAda Ada Letters, Proceedings of the 1999 Annual ACM SIGAda International Conference on Ada*, September 1999, Vol. 19, No. 3.
22. Chung, P., et. at, "DCOM and CORBA Side by Side, Step by Step, and Layer by Layer," *C++ Report*, Vol. 10, No. 1, January 1998, pp. 18-29. <research.microsoft.com/~ymwang/papers/HTML/DCOMnCORBA/S.html>.
23. DCOM Architecture, <msdn.microsoft.com/library/default.asp?url=/library/en-us/dndcom/html/msdn_dcomarch.asp>.
24. Milojevic, D., et al., "Process Migration," *ACM Computing Surveys*, Vol. 32, No. 3, September 2000, pp. 246.
25. Milojevic, D., et al., "Process Migration," *ACM Computing Surveys*, Vol. 32, No. 3, September 2000, p. 253.
26. Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Vol. 21, No. 7 July 1978, pp. 558-560.
27. Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Vol. 21, No. 7, July 1978, p. 559.
28. Raynal, M., and M. Singhal, "Logical Time: A Way to Capture Causality in Distributed Systems," *Rapport de recherche*, No. 2472, March 1995, pp. 8-9.
29. Raynal, M., and M. Singhal, "Logical Time: A Way to Capture Causality in Distributed Systems," *Rapport de recherche*, No. 2472, March 1995, p. 8.
30. Chockler, G. V.; I. Keidar; and R. Vitenberg, "Group Communication Specifications: A Comprehensive Study," *ACM Computing Surveys*, Vol. 33, No. 4, December 2001, pp. 446-449.

31. Tripathi, A., and N. M. Karnik, "Trends in Multiprocessor and Distributed Operating System Designs," *Journal of Supercomputing*, Vol. 9, No. 1/2, 1995, p. 13.
32. Agrawala, A. K., and G. Ricart, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981.
33. Lamport, L., "A New Solution of Dijkstra's Concurrent Programming Problem," *Communications of the ACM*, Vol. 17, No. 8, August 1974.
34. Agrawala, A. K., and G. Ricart, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981, p. 12.
35. Agrawala, A. K., and G. Ricart, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981, p. 13.
36. Knapp, E., "Deadlock Detection in Distributed Databases," *ACM Computing Surveys*, Vol. 19, No. 4, December 1987, p. 309.
37. Chandy, K. M.; J. Misra; and L. M. Haas, "Distributed Deadlock Detection," *ACM Transactions on Computer Systems*, Vol. 1, No. 2, May 1983, p. 148.
38. Obermarck, R., "Distributed Deadlock Detection Algorithm," *ACM Transactions on Database Systems*, Vol. 7, No. 2, June 1982, p. 198.
39. Rosenkrantz, D. J.; R. E. Sterns; and P. M. Lewis, "System Level Concurrency Control for Distributed Database Systems," *ACM Transactions on Database Systems*, Vol. 3, No. 2, June 1978.
40. Rosenkrantz, D. J.; R. E. Sterns; and P. M. Lewis, "System Level Concurrency Control for Distributed Database Systems," *ACM Transactions on Database Systems*, Vol. 3, No. 2, June 1978, pp. 186-187, 192.
41. Rosenkrantz, D. J.; R. E. Sterns; and P. M. Lewis, "System Level Concurrency Control for Distributed Database Systems," *ACM Transactions on Database Systems*, Vol. 3, No. 2, June 1978, pp. 186-197, 191-192.
42. Rosenkrantz, D. J.; R. E. Sterns; and P. M. Lewis, "System Level Concurrency Control for Distributed Database Systems," *ACM Transactions on Database Systems*, Vol. 3, No. 2, June 1978, pp. 192-193.
43. Makki, K., "Detection and Resolution of Deadlocks in Distributed Database Systems," *CIKM '95*, Baltimore, MD, 1995, p. 412.
44. Knapp, E., "Deadlock Detection in Distributed Databases," *ACM Computing Surveys*, Vol. 19, No. 4, December 1987, p. 308.
45. Makki, K., "Detection and Resolution of Deadlocks in Distributed Database Systems," *CIKM '95*, Baltimore, MD, 1995, p. 412.
46. Makki, K., "Detection and Resolution of Deadlocks in Distributed Database Systems," *CIKM '95*, Baltimore, MD, 1995, pp. 412.
47. Johnston, B.; R. Javagal; A. Datta; and S. Ghosh, "A Distributed Algorithm for Resource Deadlock Detection," *Proceedings of the Tenth Annual Phoenix Conference on Computers and Communication March*, 1991, pp. 253-255.
48. Ousterhout, J. K.; A. R. Cherenon; F. Doughs; M. N. Nelson; and B. B. Welch, "The Sprite Network Operating System," *IEEE Computer*, Vol. 21, No. 2, February 1998, pp. 23-36.
49. Doughs, E., and J. K. Ousterhout, "Process Migration in Sprite: A Status Report," *IEEE Computer Society Technical Committee on Operating Systems Newsletter*, Vol. 3, No. 1, Winter 1989, pp. 8-10.
50. Doughs, F., and J. K. Ousterhout, "Transparent Process Migration: Design Alternatives and the Sprite Implementation," *Software-Practice & Experience*, Vol. 21, No. 8, August 1991, pp. 757-785.
51. Tanenbaum, A. S., and G. J. Sharp, "The Amoeba Distributed System," <wvm.es.vu.nl/pub/amoeba/Intro.pdf>.
52. Tanenbaum, A. S.; M. F. Kaashoek; R. van Renesse; and H. E. Bal, "The Amoeba Distributed Operating System—A Status Report," *Computer Communications*, Vol. 14, No. 6, July/August 1991, pp. 324-335, <ftp://ftp.cs.vu.nl:/pub/papers/amoeba/comcom91.ps.Z>.
53. Doughs, E.; J. K. Ousterhout; M. F. Kaashoek; and A. S. Tanenbaum, "A Comparison of Two Distributed Systems: Amoeba and Sprite," *Computing Systems*, Vol. 4, No. 3, December 1991, pp. 12-14, <ftp://ftp.cs.vu.nl:/pub/papers/amoeba/cs91.ps.Z>.
54. Doughs, E.; J. K. Ousterhout; M. F. Kaashoek; and A. S. Tanenbaum, "A Comparison of Two Distributed Systems: Amoeba and Sprite," *Computing Systems*, Vol. 4, No. 3, December 1991, pp. 12-14, <ftp://ftp.cs.vu.nl:/pub/papers/amoeba/cs91.ps.Z>.
55. "The Common Object Request Broker: Architecture and Specification," *OMG Document Number 93.12.43*, December 1993.
56. Agrawala, A. K., and G. Ricart, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981.
57. Rosenkrantz, D. J.; R. E. Sterns; and P. M. Lewis, "System Level Concurrency Control for Distributed Database Systems," *ACM Transactions on Database Systems*, Vol. 3, No. 2, June 1978.
58. Johnston, B.; R. Javagal; A. Datta; and S. Ghosh, "A Distributed Algorithm for Resource Deadlock Detection," *Proceedings of the Tenth Annual Phoenix Conference on Computers and Communication March*, 1991, pp. 253-255.
59. Obermarck, R., "Distributed Deadlock Detection Algorithm," *ACM Transactions on Database Systems*, Vol. 7, No. 2, June 1982, p. 198.

The humblest is the peer of the most powerful.

—John Marshall Harlan

O! call back yesterday, bid time reeturn.

William Shakespeare—

Chapter 18

Distributed Systems and Web Services

Objectives

After reading this chapter, you should understand:

- *characteristics and examples of networked and distributed file systems.*
- *types, benefits and examples of clustering.*
- *the peer-to-peer distributed computing model.*
- *grid computing.*
- *Java distributed computing technologies.*
- *Web services and the Microsoft .NET and Sun ONE platforms.*

Chapter Outline

18.1 **Introduction**

18.2 **Distributed File Systems**

18.2.1 Distributed File System Concepts

18.2.2 Network File System (NFS)

18.3.3 Andrew File System (AFS)

18.4.4 Coda File System

18.2.5 Sprite File System

18.3 **Multicomputer Systems**

18.4 **Clustering**

18.4.1 Clustering Types

18.4.2 Clustering Benefits

18.4.3 Clustering Examples

18.5 **Peer-to-Peer Distributed Computing**

18.5.1 Client/Server and Peer-to-Peer Applications

18.5.2 Centralized vs. Decentralized P2P Applications

18.5.3 Peer Discovery and Searching

18.5.4 JXTA

18.6 **Grid Computing**

18.7 **Java Distributed Computing**

18.7.1 Java Servlets and JavaServer Pages (JSP)

18.7.2 Jini

18.7.3 JavaSpaces

18.7.4 Java Management Extensions (JMX)

18.8 **Web Services**

18.8.1 Microsoft's .NET Platform

18.8.2 Sun Microsystems and the Sun ONE Platform

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

18.1 Introduction

In the preceding chapter, we discussed the basic concepts of distributed systems, including the middleware technologies that help to build transparent and scalable systems. We also discussed how to ensure mutual exclusion in distributed systems and how to prevent and detect distributed deadlocks. In this chapter, we look at another fundamental element of distributed systems—file sharing. Accessing files stored on file servers using a distributed file system is similar to accessing files stored on the user's local computer. We discuss several distributed file systems and give an overview of how data and files are shared among distributed computers. Then, we introduce clustering, which takes advantage of distributed systems and parallel systems to build powerful computers. We discuss the peer-to-peer distributed computing model which is used to remove many central points of failure in applications like instant messengers. Another distributed computing model we discuss is grid computing, which uses unused computer power to solve complex problems. We continue our presentation of Java distributed systems technologies, with discussions of servlets, JSP, Jini, JavaSpaces and JMX, and discuss Web services and how they improve interoperability in distributed systems.

18.2 Distributed File Systems

Networked file systems allow clients to access resources stored on remote computers as the combination of a server name and the path to the resource within that server. **Distributed file systems**, discussed in detail in this chapter, are special examples of networked file systems that allow transparent access to remote files.¹ They take the form of one global file system to which all machines in the system have access. This is achieved by allowing users to mount other file systems (see Chapter 13). Once a remote file system has been mounted, the user can access the files in it as if the files were local. This section addresses the key concepts of distributed file systems and presents case studies on various key distributed file systems.

18.2.1 Distributed File System Concepts

A distributed file server can be either stateful or stateless. A **stateful** server maintains state information about client requests—such as the file name, a pointer to the file and the current position in the file—so that subsequent access to the file is faster and easier. In a **stateless** system, the client must specify this state information in each request. There are benefits to both types of systems: fault tolerance is easier to implement in stateless systems, but stateful systems can reduce the size of request messages and hence offer better performance. Distributed file systems can be characterized by transparency, scalability, security, fault tolerance, and consistency.

Transparency

Distributed file systems provide complete file location transparency; the user sees the distributed file system as a single, global file system. For example, consider a

company with offices located in Boston and New York. If a user is in Boston, the system should allow the user to access files located in either Boston or New York. However, the physical location of the files is hidden from the user. To achieve high degrees of transparency, robust naming mechanisms are required.²

Scalability

Distributed file systems should also be scalable; allowing new computers to be added to the system easily. Designers of scalable distributed file systems must consider the size of the file system, file management, data duplication and concurrent user access. Other design issues, such as narrow bandwidth, could also limit scalability. Further, if any operations are restricted to a single machine, adding clients could make that machine a bottleneck.³

Security

Two key security concerns in distributed file systems are ensuring secure communications and guaranteeing correct file-access permissions. Providing secure communications can be done by encrypting file content. Ensuring correct file access permissions can be complex. In many operating systems, the kernel can determine what files a process has access to based on the identity of the process' user. However, in a distributed system, users may have different user identifications on different machines. This can allow users to access files that they should not and prevent users from accessing files that they are authorized to access.⁴

Fault Tolerance

Fault tolerance is more complex in distributed systems than in conventional systems. When a machine crashes, the system should provide mechanisms to leave the file system in a stable and consistent state. For example, if a client requests a file and blocks while waiting for a server's response and the server then crashes, the client must have some way to guarantee that it will not wait indefinitely. Often, both the client and server will set a timer and abort a call after a timeout, but this introduces complications. What happens if a client sends a request and does not receive a response from the server? It could be that the server has crashed. It could also be that the server is slow or that its response was lost. In the latter case, if the client's request was **idempotent**, i.e., several calls to perform the same operation return the same result (such as when checking a bank balance), then the client can resend its request without introducing inconsistency into the system. However, if the client's request was not idempotent, such as when withdrawing money from a bank account, inconsistencies might arise.⁵ A well-designed distributed system makes sure that the same remote procedure calls are executed exactly once regardless of server failures.

High availability is closely related to fault tolerance. In a fault-tolerant system, the failure of a component causes no down time; a backup component immedi-

ately takes the place of the failed component and users will not be aware that a failure occurred. In a high-availability system, the failure of a component causes minimal down time. In such systems, often the backup components are used during normal execution for better performance.⁶

Consistency

Consistency is another concern for the designers of distributed systems. Many distributed systems implement **client caching** to avoid the overhead of multiple RPCs. In such cases, clients will keep a local copy of a file and flush it to the server from time to time. Because there are multiple copies of the same file, files can become inconsistent. To avoid inconsistencies, each client must **invalidate** its copy of the file against the copy on the server. The client checks the timestamp of the copy of the file on the server, if it has been updated since the client last copied the file, then the client must download the latest version. If the file has not been updated since the client copied it, then the client can work on its cached copy. Invalidation guarantees that each client will have the same view of the file, but it can lead to inefficiency if it results in the overwriting of work performed by the client. To reduce file access times, sometimes clients will also cache information about files, such as creation or modification dates and location.⁷

Servers can guarantee consistency by issuing locks to clients, giving them exclusive access to a file. When another client needs to access the file, that client must wait for the client that owns the lock to release it. The server still has ultimate control over the file and can revoke the lock if necessary. In fact, most servers **lease** locks and automatically revoke them when the leases expire. If a user still requires a lock, the user must renew the lease before it expires. In some cases, a server may even delegate complete control over the file to the client for a brief period of time. In this situation, other clients must request file access from that client instead of from the server, thereby reducing the load on the server.⁸

Designing a distributed file system involves trade-offs. One means of implementing fault tolerance is replication. But replication requires more components and introduces the problem of consistency. To ensure consistency, systems can employ client invalidation and file locks. But these increase complexity and, in some cases, restrict scalability; and these mechanisms can reduce performance.

Self Review

1. Why is fault tolerance difficult to implement in stateful systems?
2. What problem is raised by client caching?

Ans: 1) In stateful systems, if the server crashes, it must reconstruct the state information of each client when it restarts. The state information after the server restarts must be consistent with the state information before the crash. **2)** Client caching may cause file inconsistency because multiple copies of a file may be distributed to multiple clients, enabling them to modify the same file at the same time.

18.2.2 Network File System (NFS)

Sun Microsystem's **Network File System (NFS)**, the current de facto standard for network file sharing, is supported natively in most varieties of UNIX and with client and server software available for other common platforms. NFS was originally designed around TCP/IP, so it has been easily adapted to link file systems across the Internet and other wide area networks.⁹

The original version of NFS was released by Sun in 1984 and licensed free to the industry. NFS was revised in 1985 to version 2, which became integrated into various operating systems. In 1989 NFS-2 became the first version of NFS to be standardized by the Internet Engineering Task Force (IETF), becoming an Internet protocol.¹⁰ NFS was revised again with the release of version 3 in 1995.¹¹

Figure 18.1 illustrates how NFS works. Before an NFS client can access a remote file, it must use the mount protocol—a mechanism that translates a file path name to a file handle—to mount the directory that holds the remote file. A **file handle** identifies a remote file by its type, location and access permissions. Mounting involves mapping a remote file directory to a client's local directory. To mount the file system, the NFS client makes an RPC (the mount request, which contains the file path of the remote file to be mounted) to the client stub. Upon receiving the mount request, the server skeleton passes the call to the NFS server. The NFS server looks up its local file system and **exports** the local file directory that satisfies the client request, which means the server makes the local directory of files available to the remote client. The server skeleton then returns a file handle of the exported local file directory, which allows the client to access the exported file

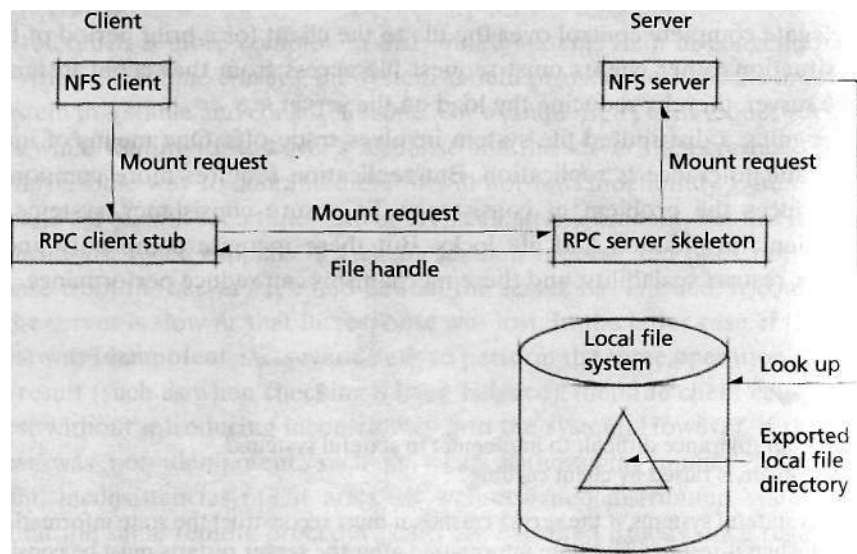


Figure 181 | NFS architecture.

directory remotely. The remote client uses this file handle in all subsequent requests to files in the exported file directory.

NFS-2 and NFS-3 assume a stateless server implementation, which makes fault tolerance easier to implement. If a stateless server crashes, the client can simply retry until the server responds. The NFS-2 and NFS-3 permission checks simply rely on the authentication information from RPC, which determines whether the client can access certain information and functions. NFS-2 uses UDP as its transfer protocol, since UDP performs well over local area networks. TCP becomes the protocol of choice on error-prone, high-latency and wide-area networks. Both NFS-2 and NFS-3 support TCP; NFS-3 optimizes its use by allowing transfers of large streams of data. Neither version 2 nor version 3 defines a caching policy to guarantee the consistency of the server and client copies of the file.¹²

Although NFS-2 and NFS-3 are structurally similar, NFS-3 introduces several key differences. First, it supports larger files—file sizes can be stored in 64 bits, while NFS-2 supports only 32-bit file sizes. Second, NFS-3 supports **safe asynchronous writes**, which allow a server to continue executing before a write has been completed. Finally, NFS-3 removes the 8KB limit defined by NFS-2 for any single request or reply.¹³ This limit was chosen as a reasonable upper bound for the size of UDP packets being transferred over local area Ethernet.

Following the release of NFS-3, Sun enhanced NFS with a service called **WebNFS**, which allows NFS clients to access WebNFS-enabled servers with a minimum of protocol overhead. WebNFS is specifically designed to take advantage of NFS-3's use of TCP to support efficient file operations over the Internet.

Many features provided by WebNFS have been integrated into version 4 of NFS, which entered development in 1999 as an IETF proposal. NFS version 4 maintains most of the features of prior versions of NFS and provides efficient operation over wide area networks, strong security implementation, stronger interoperability and client caching.¹⁴

Like its predecessors, NFS-4 uses RPCs to communicate between client and server. NFS-4 offers a new feature, allowing multiple related RPCs to be packaged together into one request. This limits network traffic and increases performance. Also, NFS-4 is stateful, which allows faster access to files. With earlier versions of NFS, the client used the mount protocol to receive the file handle. In NFS-4, the client obtains the file handle without the mount protocol. The client gets a file handle referring to the root of the file system on the server, then traverses the file system to locate the requested file. By removing the mount protocol, the client does not need to mount the different file systems of a server separately.¹⁵

NFS-4 extends the client-caching scheme through **delegation**, whereby the server temporarily transfers control of a file to a client. When the server grants a read delegation, no other clients can write to that particular file but they can read from it. When the server grants a write delegation, then no other clients can read from or write to that particular file. If another client requests write access to a file with the read or write delegation, or the client requests read access to the file with

write delegation, the server will revoke the delegation and request that the original client flush the file to disk.¹⁶

As discussed earlier, the only security provided by previous versions of NFS was weak authentication from RPC. The developers of NFS-4 were able to take advantage of new RPC security which also supports several cryptographic algorithms which are used when NFS-4 sets up communication sessions and encrypts each RPC.¹⁷

Self Review

1. Does NFS-2 require stateful server implementation? What is the benefit of such a design?
2. When an NFS-4 client owns the read delegation of a file, can other clients read or write to that file?

Ans: 1) No, NFS-2 servers must be stateless. With a stateless server, it is easy to implement fault tolerance. 2) Other clients can read from, but not write to, the file.

18.2.3 Andrew File System (AFS)

Development of the **Andrew File System (AFS)**, a scalable distributed file system, began in 1983 at Carnegie Mellon University. The goal was to provide a distributed file system that would grow to support a large user community while being secure. The designers hoped to exploit advances in workstation processing capabilities to facilitate scalability, efficiency and security. AFS is a global file system that appears as a branch of a traditional UNIX file system at each workstation, it is based on the client-server model and relies on RPCs for communication. Andrew went through three major revisions: AFS-1, AFS-2 and AFS-3.¹⁸⁻¹⁹

The entity that governs distributed file access in AFS is called **Vice** (Fig. 18.2). Vice processes run on distributed file servers and interact with a user-level process called **Venus** at each client. Venus interacts with the kernel's **virtual file system**

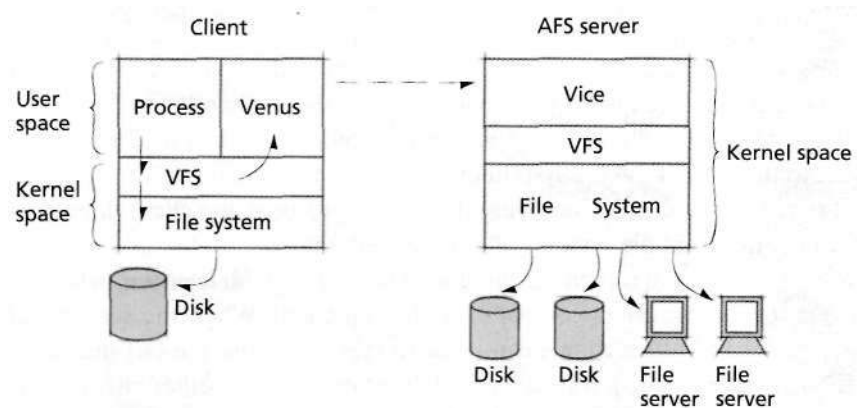


Figure 18.2 | *AFS structure.*

(VFS), which provides the abstraction of a common file system at each client and is responsible for all distributed file operations. AFS obtains scalability from its caching method. The local **nonvolatile storage** (disk) at each workstation serves primarily as a cache to the distributed file system. The data stored in nonvolatile storage is not lost when a machine loses power. When a user process tries to open a file, it makes a request to the VFS which consequently checks to see if the file is in the cache. If it is, the VFS simply accesses the file. If not, the VFS communicates through Venus to Vice, which retrieves the file from the correct server.²⁰

In AFS-1 and AFS-2, the cache was updated only at the file level—block updates were not possible. This guaranteed that users working locally on a file would not have to wait long for network access once the file was open. AFS-3 was also upgraded to support block-level caching, which allows large files to be manipulated by clients with small caches.

In AFS-1, each time the VFS tried to open a cached file it would have Venus contact the server to make sure that the file was still valid. This generated many network calls, even when files had not been modified. In AFS-2 and AFS-3, the server becomes responsible for ensuring each client has the most up-to-date copy of its files. Each server keeps track of the files that clients cached. Each time a client closes a file, Venus flushes it to Vice, so the server always keeps the most up-to-date copy of the files. When a file is flushed to Vice, the server performs a **callback** to clients that have that file cached. When a client receives a callback, it knows that its cached version is no longer valid and requests the newest version from the server.

In AFS-1 the server had one process listening on a dedicated port for connection requests. Upon connection, this process would fork another process to handle the Vice functions for the duration of the client's session. Because processes on different machines cannot share main memory, all shared data had to be written to the server's local file system. Furthermore, servers suffered performance problems due to the time associated with the context switches that were required to service different clients. In AFS-2 and AFS-3 there is one dedicated process that services all client requests. This process spawns a separate thread for each client and allows all state information to be loaded into virtual memory. Because the state information tends to be small for each client, it can generally be held in main memory. This eliminates context switches and allows interthread communication through shared memory. The RPC mechanism is incorporated into the thread package to further improve performance.²¹

The overall AFS structure also changed with each revision. In AFS-1 different subtrees were divided among different servers, which were known as the custodians. Each server also had a directory that would allow Vice to direct requests for different subtrees to different servers. Venus located resources based on pathname. This had several drawbacks—resolving an entire path could generate multiple calls to Vice and could span multiple custodians. AFS-2 introduced **volumes** to manage subtrees. Volumes are primarily of administrative value, allowing replication and isolation of certain subtrees, and are transparent to users. Generally each user is

assigned a volume. Making the idea of volumes concrete also allowed the implementation of **file identifiers (fids)** in Venus. A fid specifies a volume, an index within a volume and an identifier to guarantee object uniqueness within a volume. The AFS-2 implementation of Venus resolved filenames in this manner, which simplified file lookups and allowed the renaming of directories. Each server had a directory to allow it to find volumes on any other server. AFS-3 was designed to be a large distributed file system. In fact, one continuous AFS-3-based namespace spanned multiple geographically distant universities in the United States. Obviously, there cannot be centralized administration of such a file system as each university typically has its own managerial requirements. Therefore, AFS-3 introduced **cells**, which preserve namespace continuity while allowing different systems administrators to oversee each cell.^{22, 23}

The security of AFS is much stronger than that of earlier versions of NFS, partially due to the underlying RPC structure. Because AFS is designed to grow to service many computers, its designers decided that it would not be safe to trust the network or any client. Therefore, every RPC is encrypted and must be authenticated. In AFS-1 and AFS-2 the permissions of a given user are related to the permissions of each group to which the user belongs. In AFS-3 permissions must be specified individually. Upon system login, users must specify a password. Vice sends the encrypted password to an authentication server, which responds with a token. Secure RPCs use these tokens to provide secure communication. File access is governed by access control lists (ACLs). Section 19.4.3, Access Control Mechanisms, discusses ACL in detail. Each user is allowed to perform certain operations on certain files. Authentication is guaranteed by matching the credentials on each token to the permitted operations in the ACL.²⁴

18.2.4 Coda File System

AFS is scalable, indeed able to service thousands of computers, but it has one large drawback—it is not fault tolerant. The crash of one server or disruption of network service can render hundreds of computers inoperable. To deal with this, the designers of AFS began research on Coda in 1987.²⁵

Coda is derived from AFS-2. As in AFS-2, Coda uses local nonvolatile storage (e.g., disks) as file caches, and Vice uses callbacks to keep clients up to date. Volumes are the basic division in the file system and Coda performs file lookups in the same way as AFS-2. The goal of fault tolerance led to the addition and modification of several of AFS's features, including the idea of a volume.²⁶

In Coda, volumes are logical pieces of the file system and are replicated physically across multiple file servers. Servers that hold the same volume are known as a **volume storage group (VSG)**. Although the multiple servers are transparent to users, the Venus process on each client periodically probes the VSGs of each of its volumes. The members of the VSG with which the client can connect and communicate are known collectively as the client's **available volume storage group (AVSG)** (Fig. 18.3). Coda uses AVSGs to implement fault tolerance.²⁷

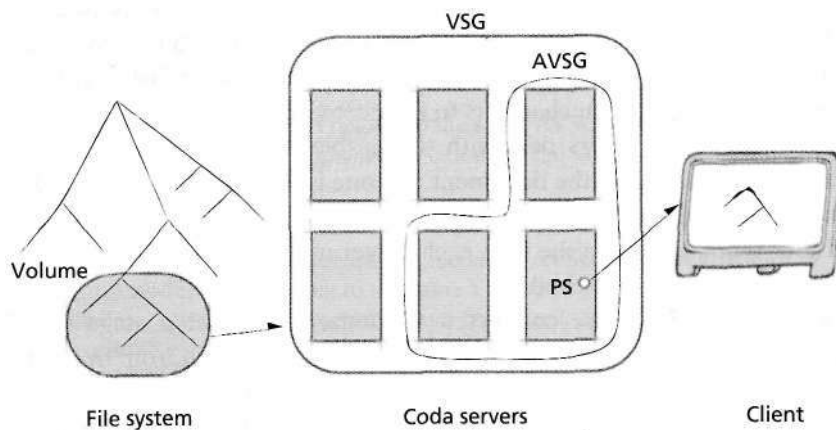


Figure 18.3 | Coda volume structure.

When a client process tries to read a file, its request is directed to the local VFS. If the file is in cache, then the read continues as in AFS-2. Otherwise, Venus contacts a member of the AVSG known as the **preferred server** (PS). Venus downloads a copy of the file from the PS and downloads file version information from each member in the AVSG. If the versions on each server agree, then the client is allowed to use the file. If the versions from different servers do not agree, then the calling process aborts and the outdated copies are updated. After the members of the AVSG have been updated, the requesting client downloads a copy of the file. Note that if the PS or any member of the AVSG crashes, the client can still operate, whereas in AFS-2 the client would be stuck.²⁸

Upon closing a file, Coda clients write a copy of the file to each member of the AVSG. The write occurs in two steps using the **Coda Optimistic Protocol**. The first step, COP1, writes the file as we have discussed. The second step, COP2 writes an **update set**, which specifies the members of the AVSG that have successfully performed the COP1, to each of the members of the AVSG. If any of the members are **not** in the update set, it is assumed that they have not updated (even if they have and their reply was lost) and this is noted to remind the system to update them the next time communication is established. To decrease write delays, COP2 messages are performed asynchronously and are appended on future COPf messages. If other clients have a copy of the file in cache, then servers will have to perform callbacks.²⁹

These operations will provide a consistent view of a file within an AVSG. When a file is opened, it will be the same as the last copy that was written to Coda. However, Coda's replication scheme is optimistic, which means it assumes that files in different AVSGs are consistent and it allows clients to access and modify such files. This guarantees that files will always be available, but it might lead to inconsistencies and the loss of work.

We now discuss Coda's replication scheme and potential consistency issues. Consider a VSG and two clients whose AVSGs are mutually exclusive (Fig. 18.4). It is possible for each client to download, modify and upload the same copy of a file to its AVSG. Now the view of that file within the VSG will not be consistent.

Coda provides some mechanisms to fix inconsistencies. If the file in conflict is a directory, the Coda servers deal with the problem. Each member of the VSG locks the volume that holds the document and one is designated the leader. Logs of the transactions of each server are sent to the leader, which combines the logs and redistributes them. Based on the logs, each server updates its view of the directory. Finally, the volumes are unlocked.

When Coda clients are connected to Coda, they enter a stage called the **hoarding stage**, where they prepare for a possible disconnection from the system by caching any requested files. When the client becomes disconnected, it enters the **emulation stage**, during which all file read requests are serviced from the cache. The files could be stale, but the system is still operable. If the user tries to access a file that is not in cache, the system reports an error. Writes during the emulation stage occur in two steps. First, the file is updated on disk. Second, the **client modification log (CML)** is updated to reflect file changes. Venus modifies the CML so that when a log entry is undone by a later entry, the earlier log entry is removed. The CML also facilitates updates during the **reintegration stage**. Reintegration occurs immediately after the client reconnects to the system when Venus asynchronously updates the server using the CML.³⁰

A client may become disconnected for several reasons: All of the servers in a VSG could crash, a network connection could be faulty or the user could intentionally disconnect from the network. Coda handles each of these cases in the same way, and the transition between connected and disconnected states is transparent.

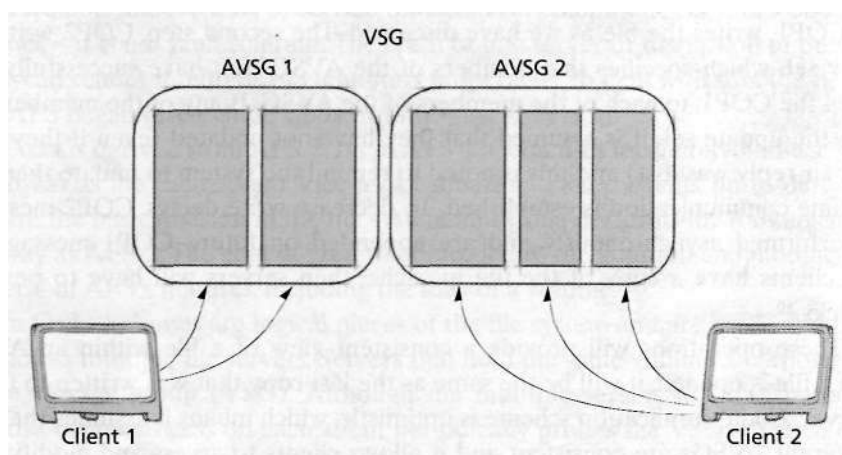


Figure 18.4 | File consistency issues in mutually exclusive AVSGs.

This makes Coda particularly attractive for the growing number of mobile computers, and it is one of the primary reasons that Coda is still undergoing improvement.

Self Review

1. In Coda, if client A and client B belong to the same AVSG, does it imply that they belong to the same VSG? If client A and client B belong to the same VSG, does it imply that they belong to the same AVSG?
2. What is the problem with Coda's replication scheme?

Ans: 1) If client A and client B belong to the same AVSG, then they belong to the same VSG. If client A and client B belong to the same VSG, then they may not belong to the same AVSG. 2) Coda's replication scheme provides a consistent view of a file within an AVSG but does not guarantee consistency within the VSG. Coda's replication scheme assumes that files in different AVSGs are consistent and allows clients to access and modify such files within different AVSGs, which causes inconsistency within the VSG.

18.2.5 Sprite File System

In Chapter 17, we overviewed the Sprite operating system. This section focus on the Sprite file system. A primary goal for the Sprite file system was transparency. Sprite's developers attained a better level of transparency than that offered by NFS, AFS or Coda. A second goal was to implement complete consistency.³¹

The Sprite file system has the perceived form of a UNIX file system. Unlike NTS or Coda, however, it gives every client the exact same view of the hierarchy. The Sprite file system also goes one step further in UNIX file system emulation by allowing transparent remote access to I/O devices (which are represented as files in UNIX). Although hidden from the user, the Sprite file system is divided into **domains**. Each domain represents a piece of the global file hierarchy and is stored at one server (Fig. 18.5). Except for the domain at the top of the global file hierarchy, the root of one domain is the deepest child of another.³²

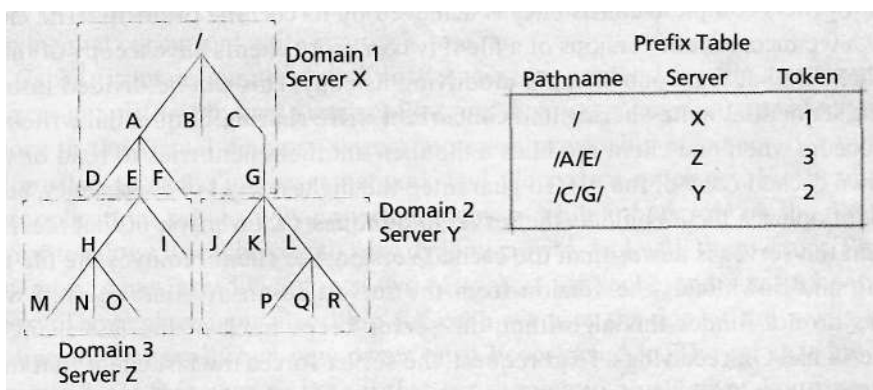


Figure 18.5 | Sprite file system domains.

The kernel of each client keeps a private copy of a **prefix table**, which stores domain information to aid in file lookups (Fig. 18.5). Each entry in the table represents a separate domain and consists of the absolute path to the root directory within the domain, the server that houses the domain and a token that identifies the domain. To look up a file, a client takes an absolute pathname and locates the longest matching prefix within the prefix table. If no such prefix can be found, the client broadcasts a request to every server. The server that houses the domain that matches the path responds with information to populate the client's prefix table, and the client continues. The client then removes the matched prefix from the name and sends what is left to the indicated server along with the domain identification token. The server resolves the name within the specified domain and returns a token pointing to the requested file. If the resolution cannot be completed because the path spans multiple domains, the server returns the pathname of the new domain and the client repeats the process. Sprite processes that access files keep the server address and token identifying the domain of the working directory as part of their state. Processes that access files via relative paths provide this state information in addition to the relative path to allow servers to resolve the entire path.³³

In Sprite, both the client and the server keep a cache in main memory. To open a file, the client first checks its cache. If the file is not in the client's cache, the client then makes a request to its backing store (which is generally a server). If the server is unable to satisfy the request from its cache, it reads the data in from disk (Fig. 18.6, step a). The server then stores the data in its cache and passes the data to the client. The client retains a copy of the data in its cache as well (Fig. 18.6, step b). When a client writes a block, it writes into its cache. Sprite flushes updated pages to the server every 30 seconds though the cache management algorithm may choose to flush the page sooner. The page will then remain in the server's cache for 30 seconds before it is flushed to disk (Fig. 18.6, step c).

The lazy write-back strategy was chosen to trade an increase in performance for a decrease in fault tolerance. As in UNIX systems, modified pages in a cache will be lost during a crash if they have not been flushed to the server.³⁴

Sprite's complete consistency is achieved by its caching protocols. The clients may have inconsistent versions of a file if two or more clients have a copy of the file in cache and at least one client is modifying its copy. This can be divided into two cases: **sequential write-sharing** and **concurrent write-sharing**. Sequential write-sharing occurs when one client modifies a file then another client tries to read or write its own cached copy of the file. To guarantee the highest level of consistency, before a client opens a file it contacts the server and requests the version of that file. If the file on the server is newer than the cached version, the client removes the file from cache and downloads the version from the server. To ensure that delayed write-backs do not hinder this algorithm, the server keeps track of the last client that writes a file. On receiving a read request, the server forces the last client that modifies the file to flush all modified pages to disk immediately, before servicing the request. Concurrent write-sharing occurs when two clients modify cached copies of

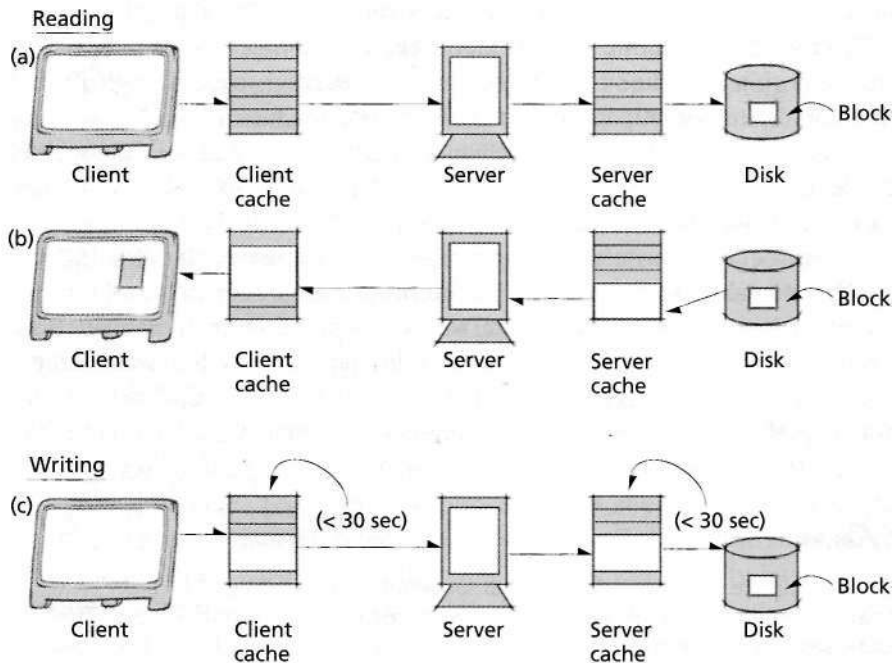


Figure 18.6 | Reading and writing in the Sprite file system.

the same file at the same time. To ensure consistency, Sprite disables caching of that file for all clients. Reads and writes are sent directly to the server, and the state of the file therefore appears the same to all clients.

Note that although each client will have a consistent view of the file, there could be synchronization issues associated with this algorithm. Sprite provides locking system calls for applications that cause the inconsistency. This solution also hinders file system performance. The developers of Sprite justified their implementation by observing that concurrent write-sharing is rare.³⁵

Cache (in main memory) and virtual memory contend for the same system resources in Sprite—physical memory. Whereas devoting a large portion of physical memory to the virtual memory system increases processing efficiency, devoting a large portion to cache increases network and file system efficiency. Furthermore, some applications rely heavily on virtual memory while others rely on the file system. Sprite allows its cache size to vary dynamically to deal with these issues. Physical memory pages are divided into two sets, a set for cache and a set for virtual memory. The replacement algorithm for each set uses the age of the pages as a guideline for removal when new pages must be swapped in. The set that has the older page loses that page to the set that is swapping in a new page. For example, consider the situation where 16 pages of physical memory are devoted to cache, 16 pages are devoted to virtual memory and one of the virtual memory's pages is the

oldest of all 32 pages. If a process tries to bring a page into cache, the oldest virtual memory page is removed and the cache is then allocated 17 total pages.

This introduces a subtle point of inefficiency. What happens if a page is in both virtual memory and cache? Clearly the system is wasting space. To avoid this, virtual memory writes directly to the server when updating files.

We have seen that Sprite is more concerned with performance than with fault tolerance. This point is strengthened when considering how Sprite deals with server crashes. Sprite does not use replication servers. Instead, in the hope that crashes would be infrequent, Sprite's designers implemented a server that would recover quickly from crashes. To do this they placed the burden of state control on the clients, using a log-structured file system with checkpointing on the server (as discussed in Section 13.10.2). The motivation for log-based file system was to improve I/O performance. When clients realize that a server has crashed, they wait until it reboots and then send it information about its files that they have cached. The server uses this information to rebuild its state.³⁶

Self Review

1. How is complete consistency achieved in sequential write-sharing?
2. What problem might result from placing the burden of state control on the clients? Propose a solution to this problem.

Ans: 1) Sequential write-sharing occurs when one client modifies a file and then another client tries to read or write its own cached copy of the file. Before a client opens a file, it contacts the server and requests the version of that file. If the file on the server is newer than the cached version, the client removes the file from cache and downloads the version from the server. To ensure complete consistency, the server keeps track of the last client that writes a file. On receiving a read request, the server forces the last client that modifies the file to flush all modified pages to disk immediately, before servicing the request. **2)** All of the clients will bombard the server with information when it reboots, possibly swamping the network. Sprite's developers built a negative acknowledgement into their implementation. This allows the server to tell the client that it is busy now, but has not crashed and to come back later.

18.3 Multicomputer Systems

As discussed in Chapter 15, multiprocessing is the use of multiple processors to execute separate portions of a computation in parallel. Several classes of machines are generally referred to as multiprocessors, but the most popular are machines that have similar or identical processors and that share a common memory. In contrast, **multicomputer** systems do not share a common memory or bus (although they may share a virtual memory). Each processing unit has access to its own resources. These independent units are connected in a network to operate cooperatively to form a multicomputer system. Multicomputer systems can be either homogenous or heterogeneous. A homogeneous multicomputer system consists of computers that have the same hardware, run the same operating systems and communicate across the same network architectures. A heterogeneous multicomputer system consists of

computers that may have different hardware, run different operating systems and communicate across different network architectures.³⁷

Modern supercomputers are massively parallel processing (MPP) machines capable of hundreds of gigaflops or even teraflops. A flop is a floating point operation per second.³⁸ Currently under construction at Los Alamos is the supercomputer Q which will top 100 teraflops. It will be used by scientists to simulate the effects of aging on America's nuclear arsenal, using three-dimensional models of explosions.³⁹ Los Alamos scientists are also working on a low-end supercomputer called Green Destiny built from off-the-shelf components. Green Destiny provides 160 gigaflops of computing power using 100Base-T Ethernet and 240 667-megahertz Transmeta Corp. processors.⁴⁰ Green Destiny is a multicomputer system.

Another method of implementing a multicomputer system is with grid computing which uses middleware that can run like an application on a general-purpose computer. Distributed.net pioneered academic distributed computing over the Internet by having users download an application which performed operations on their own computers.⁴¹ Today there is also SETI@Home, which uses volunteers' computers to process data collected by the Search for Extraterrestrial Intelligence project. Intel, United Devices, and the American Cancer Society have created a screensaver software that performs calculations to aid in pharmaceutical research.⁴²

Companies such as United Devices are developing software and hardware packages to create enterprise computer grids which will allow organizations to leverage unused processor cycles on their networks to handle computer-intensive tasks. We discuss such grid computing in Section 18.6, Grid Computing.

Distributed processing makes large computations feasible. However, this power comes at the cost of simplicity. Just as in uniprocessing environments, distributed processing must take into account synchronization, mutual exclusion and deadlock. Networking and the lack of shared memory make managing such issues more complex.

Self Review

1. Distinguish multiprocessing and multicomputer systems.
2. What makes distributed processing difficult to implement?

Ans: 1) With multiprocessing, processors share a common memory. Multicomputer systems do not share a common memory—each processing unit has access to its own resources. 2) Distributed processing must take into account synchronization, mutual exclusion and deadlock. Networking and the lack of shared memory make managing such issues more complex.

184 Clustering

Recall that in a distributed system, several computers handle user requests simultaneously. Each computer processes user requests independently. However, the distributed system must handle communication delays introduced by its underlying networks. Multiprocessing fixes the communication delays by allocating many pro-

processors for one computer, making the communications among processes easier and faster. Yet, multiprocessing has its own disadvantages: high cost/performance ratio, limited scalability and not all types of applications running on a system benefit from multiprocessing.⁴³

Clustering—interconnecting nodes (single-processor computers or multiprocessor computers) within a high-speed LAN to function as a single parallel computer— is architecturally intermediate between distributed computing and multiprocessing. The set of nodes that forms the single parallel machine is called a **cluster**. Clustering enables multiple computers to work together to solve large and complex problems.⁴⁴

Self Review

1. What is clustering?
2. Why would clustering be employed?

Ans: **1)** Clustering interconnects nodes within a high-speed LAN to function as a single parallel computer. **2)** Clustering takes advantage of multiprocessing and distributed systems while eliminating the communication delays associated with fully distributed systems, providing a powerful computer that is capable of solving large and complex problems.

18.4.1 Clustering Types

There are three major types of clusters. In a **high-performance cluster**, all of the nodes in the cluster are used to perform work. In a **high-availability cluster**, some of the nodes perform work while others serve as backups. If working nodes or their components fail, the backup nodes start running and take over the jobs that were being executed on the failed nodes immediately without interrupting service. In a **load-balancing cluster**, a particular node works as a **load balancer** to distribute the load (such as thousands of requests from the clients) to a set of nodes so that all hardware is efficiently utilized.⁴⁵

High-performance clusters are used for solving large-scale computations that can be divided into smaller problems to be solved in parallel. High-availability clusters are used for mission-critical applications in which failure must be avoided. Load-balancing clusters benefit organizations that handle large volumes of user requests. For example, high-performance clusters are useful for analyzing images from Magnetic Resource Imaging (MRI) scanners, while high-availability clusters are crucial in aircraft control systems and load-balancing clusters are usually used in e-business applications with high user volumes.

Self Review

1. Does a high-availability cluster require that its nodes be working all the time?
2. Argue why a cluster for a particular type of application should have the attributes of all three types of clusters. Consider a search engine being used for an application where lives could be at stake, such as locating organs for transplant.

Ans: **1)** No. High-availability clusters require that only some of their nodes be working while others act as backups. **2)** The cluster should be high performance, because billions of

Web pages would have to be searched in a fraction of a second. It should be high availability to ensure that it is always ready to operate when needed, especially in an emergency. It should be load balancing to spread the load in peak periods, ensuring that individual requests are not delayed.

18.4.2 Clustering Benefits

There are several benefits associated with clustering. It economically interconnects relatively inexpensive components. This reduces the cost for building a clustered system compared to a single parallel computer with the same capability.⁴⁶ High performance is another advantage of clustering. Each node in the clustering system shares the workload. Communications among nodes in the cluster are faster than those in unclustered distributed computing systems due to the high-speed LAN between nodes.⁴⁷ Clustering can also provide replication of resources across the nodes, so that the failure of any one computer will not affect the availability of the system's resources. If a service on one node is down, other nodes that provide the same service in the cluster can still function and take over the tasks performed by the failed component.⁴⁸ Clustering inherits the scalability of distributed systems. A cluster is able to add or remove nodes (or the components of nodes) to adjust its capabilities without affecting the existing nodes in the cluster. Clustering provides better scalability than multiprocessing. For example, the HP 9000 Superdome can have up to 64 processors, while the HP XC clusters can scale up to 512 processors.^{49,50} Clustering achieves reliability and fault tolerance by providing backups for the services and resources. When failure occurs in one running service, the cluster immediately switches the process to the backup service without affecting the performance and capacity of the system, providing uninterrupted service.⁵¹

Self Review

1. Why is communication among nodes in a cluster faster than among nodes in an unclustered distributed computing system?
2. How does clustering implement fault tolerance?

Ans: 1) The reason is that a high-speed local area network is generally faster than the wide area networks used in distributed systems. 2) Clustering provides backups for services and resources.

18.4.3 Clustering Examples

In this section, we consider how clustering is implemented in Linux and Windows. Many Linux clustering solutions are available, the best known being **Beowulf**, a high-performance cluster.⁵² In 1994, the NASA Earth and Space Sciences (ESS) project built the first Beowulf cluster to provide parallel computing to address large-scale problems involved in ESS applications. Figure 18.7 shows a typical Beowulf cluster. A Beowulf cluster may contain up to several hundred nodes. Theoretically, all nodes have Linux installed as their operating system and they are interconnected with high-speed Ethernet (often with a bandwidth of 100 Mbps). Each

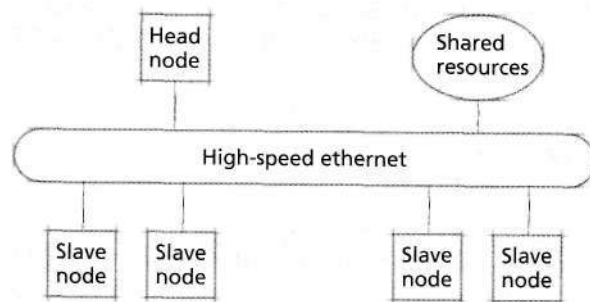


Figure 18.7 | Typical Beowulf cluster.

Beowulf cluster has a **head node** (also called a **master node**) that acts as a server to distribute the work load, control access to the cluster and handle the shared resources. All other nodes in the cluster are often called **slave nodes**. The head node usually has a monitor and keyboard, while slave nodes do not. All slave nodes should be configured with the same processor, memory and disk space so that all nodes can complete the same job at approximately the same time. Usually, all the nodes in the cluster are connected within a single room to form a supercomputer.⁵³

Windows Server 2003 can be used to build both high-availability clusters and load-balancing clusters. A high-availability cluster built with Windows Server 2003 can have at most eight nodes. All its nodes may share a storage device (Fig. 18.8) or each node may have a local storage, in which case the backup node must keep the exact same data (Fig. 18.9). Unlike the Beowulf cluster, the nodes in the high-availability Windows cluster can be connected either in a LAN or a WAN. In the case of shared storage, only one node is running at a time and that node has the control of the shared storage. When the running node fails, a backup node takes over control of the shared storage. However, if the shared storage breaks down, the entire clus-

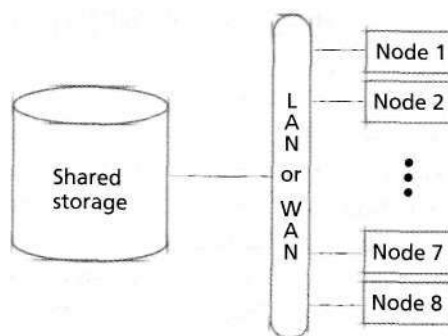


Figure 18.8 | High-availability cluster with shared storage.

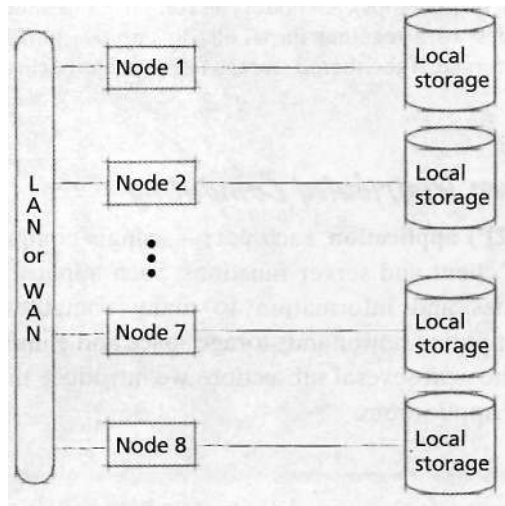


Figure 18.9 | High-availability cluster with local storage.

ter cannot keep running. When each node has its own storage, more than half of the cluster nodes must be active at all times so that when one or more nodes fail, other nodes can continue work effectively. For example, if the cluster has eight nodes, at least five of them must be active and ready to function.⁵⁴

A load-balancing cluster built with Windows Server 2003 can have at most 32 nodes (Fig. 18.10). The cluster does not require sharing storage, because each node can do its job independently. If one node fails, the remaining nodes in the cluster are still capable of handling user requests. Scalability is achieved with the load-balancing cluster, because it is easy to add or remove a node from the cluster. Usually the nodes in the load-balancing cluster are interconnected with high-speed Ethernet.⁵⁵

Self Review

1. List three characteristics that make Beowulf clusters high-performance.
2. What problem does the high-availability cluster with shared storage have?

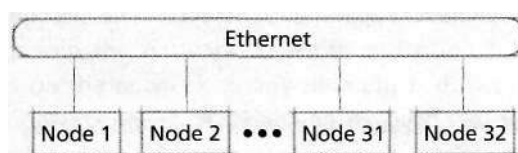


Figure 18.10 | Load-balancing cluster with 32 nodes.

Ans: 1) All nodes in the Beowulf cluster are interconnected with high-speed Ethernet (often with a bandwidth of 100 Mbps). All nodes can complete the same job approximately at the same time. All nodes work together in parallel to improve performance. **2)** All nodes depend on the shared storage. If the shared storage fails, the entire cluster fails.

18.5 Peer-to-Peer Distributed Computing

In a **peer-to-peer (P2P) application**, each **peer**—a single computer in the P2P system—performs both client and server functions. Such applications distribute processing responsibilities and information to many computers, thus reclaiming otherwise wasted computing power and storage space and eliminating many central points of failure. In the next several subsections we introduce the fundamental concepts of peer-to-peer applications.

18.5.1 Client/Server and Peer-to-Peer Applications

Many network applications operate on the principle that computers should be segregated by function. Some computers, such as servers, offer common stores of programs and data. Other computers, such as clients, access the data provided by the servers. Using a search engine from a Web browser is an example of a client/server application. Clients send queries to servers, which access various databases and respond with the requested information.

P2P applications are different from client/server applications because all computers in the network act as both clients and servers. Each peer has the ability to discover and communicate with other peers and may share resources (such as large multimedia files) with others. P2P applications are similar to the telephone system—a single user can both send and receive information.⁵⁶

Self Review

1. How do client/server applications work?
2. List the advantages of peer-to-peer applications over client/server applications.

Ans: 1) In a client/server application, the client sends requests to the server, and the server processes the client request, often accessing one or more databases, and sends the response back to the client. **2)** It is easier to set up the network for a peer-to-peer application, and no network administrator is required. In a peer-to-peer application, if one peer fails, others can still function well. In a client/sever application, if the server fails, the entire application fails.

18.5.2 Centralized vs. Decentralized P2P Applications

P2P applications can be implemented in two forms: centralized and decentralized.⁵⁷ A **centralized P2P application** uses a central server system that connects to each peer. Centralized P2P applications are similar to the client/server relationship. In a

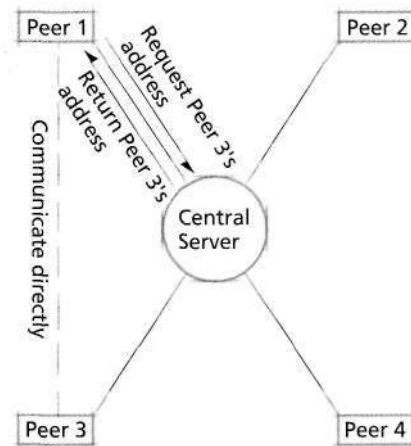


Figure 18.11 | Centralized P2P instant-messaging application.

centralized instant messaging application (Fig. 18.11), when peer 1 wants to talk to peer 3, peer 1 must get the address of peer 3 from the server first. Then peer 1 communicates with peer 3 directly. One major weakness of this centralized system is the dependency on the server. If one or more central servers fail, the entire network could fail. The server's capabilities limit the application's overall performance. For instance, Web sites can fail when malicious users overload the Web server(s) with an excessive number of requests. However, centralized architectures also have advantages, such as simplifying management tasks (e.g., monitoring user access by providing single points of network control).

A **pure P2P application**, also called a **decentralized P2P application**, does not have a server and therefore does not suffer from the same deficiencies as applications that depend on servers. In a pure P2P instant messenger application (Fig. 18.12), when peer 1 wants to send a message to peer 3, peer 1 no longer needs to communicate with the server. Instead, peer 1 discovers peer 3 via distributed search mechanisms and sends messages to peer 3 directly. If a peer in a well-designed, pure P2P system fails, all other peers continue to function, so such systems are particularly fault tolerant.

Peer-to-peer applications have disadvantages as well. Anyone with the appropriate software can join the network of peers and often remain anonymous, so determining who is on the network at any moment is difficult. Also, the lack of a server hinders the protection of copyright and intellectual-property rights. Real-time searches can be slow and increase network traffic, as queries propagate throughout the network.⁵⁸ Figure 18.13 lists some common peer-to-peer applications.

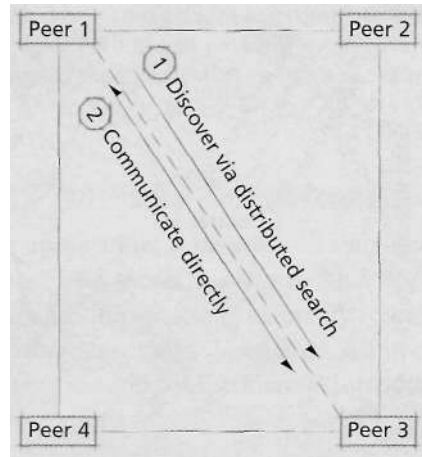


Figure 18.12 | *Pure P2P instant-messaging application.*

<i>Distributed Application</i>	<i>Description</i>
Gnutella	A P2P technology used to share documents on the Internet. Gnutella does not use any servers. There is no authentication, and peers search for files via a distributed search mechanism. ⁵⁹ (Section 18.5.3, Peer Discovery and Searching, overviews this mechanism.)
KaZaA	A file sharing application that is a hybrid between Gnutella and centralized applications. A server authenticates all users. Certain peers serve as search hubs, which catalog the files of peers connected to them. Searches are distributed to each search hub, which then responds with results that allow direct connections for file transfers. ⁶⁰
Groove	A P2P system that allows users to communicate, collaborate and share documents on the Internet and intranet. Groove provides secure communication because users are authenticated and private data is not shared with third parties. ⁶¹
Freenet	Decentralized P2P technology that allows users to share documents on the Internet without fear of censorship. Freenet does not have a central server to govern access. Instead, access to Freenet is anonymous. Documents stored in Freenet are encrypted to improve protection. ⁶²
Instant messaging	P2P application that enables users to send short text messages and files to one another. Most instant messengers use servers that authenticate all users and route messages between peers.

Figure 18.13 | *Common P2P applications.*

Pure peer-to-peer applications are completely decentralized. Some file sharing applications are not pure peer-to-peer, because they use servers to authenticate users and index each peer's shared files. However, peers connect directly to one

another to transfer files. In such a system, centralization improves search performance but makes the network dependent on a server. Performing file transfers between peers decreases the load on the server.

Self Review

1. What are the advantages of centralized P2P applications?
2. What is a benefit of a decentralized P2P application over a centralized one?

Ans: 1) Centralized P2P applications simplify management tasks, such as monitoring user access by providing single points of network control. 2) Centralized P2P applications rely on a server for certain operations. If the server fails, those operations will be unavailable.

18.5.3 Peer Discovery and Searching

Peer discovery is the act of finding peers in a P2P application. Pure P2P applications often suffer slow peer discovery and information searching due to the lack of a server. There is no general solution to peer discovery. Gnutella presents one approach for circumventing these problems. Gnutella is a pure peer-to-peer technology that enables distributed information storage and retrieval. Users can search for and download files from any peer on the network. Users first join a Gnutella network by specifying the network address of a known Gnutella peer. Without knowing at least one peer on the network, a user cannot join the network. Each user's Gnutella software functions as a server and uses the HTTP protocol to search for and transfer files.⁶³

To perform a search, a peer sends search criteria to the peers connected to it. Those peers then propagate the search throughout the network of peers. If a particular peer can satisfy the search, that peer passes this information back to the originator. The originator then connects directly to the target peer and downloads the information. The peer that made the original query can identify itself only when it connects directly to the peer with the requested file to begin file transfer.

In the Gnutella network, a peer forwards a search request to all of its directly connected peers in parallel. For example, if P_1 receives a request from the user, P_1 forwards the search request to all its directly connected peers first. These peers then work in parallel; each one tries to fulfill the search request and also forwards the request to its directly connected peers.

In the Freenet P2P application, each peer forwards a search request to a single directly-connected peer at a time. For example, if P_1 receives a request from the user, P_1 forwards the search request to P_2 which is one of its directly connected peers. If P_2 cannot answer the request, it forwards the request to one of its directly connected peers. If all the nodes that P_2 can reach cannot answer the request, P_1 then forwards the search request to another one of its directly connected peers.⁶⁴

Searches conducted in both Gnutella and Freenet are called **distributed searches**. Distributed searches make networks more robust by removing single points of failure, such as servers. Not only can peers find information in this way, but peers can search for other peers via distributed searches as well.

Self Review

1. What are some of the problems with distributed searches?
2. Compare Gnutella's distributed search mechanism to Freenets.

Ans: 1) Distributed searches are costly, possibly generating enormous amounts of network traffic. If poorly implemented P2P systems proliferate, this could have serious ramifications on the performance of the entire Internet. Distributed searches do not guarantee to find the requested files. 2) Searches in Gnutella will reach close peers before reaching any more distant peers. In Freenet, a search can reach a distant peer before all close peers have been searched.

18.5.4 JXTA

Sun Microsystems, Inc., created Project JXTA (short for Juxtapose) in response to the growing popularity of peer-to-peer applications. Project JXTA strives to create a standard, low-level, platform and language-independent protocol that promotes interoperability among peer-to-peer applications. The current JXTA implementation is written in Java, but developers can implement JXTA in any programming language. JXTA provides a foundation on which developers can build any type of P2P application.

JXTA attempts to solve the following problems of peer-to-peer applications:⁶⁵

1. Security/authentication—Large peer-to-peer network applications, such as AOL Instant Messenger (AIM) and MSN Instant Messenger, use servers to bootstrap users onto the network.⁶⁶ This bootstrapping ensures that the same person uses a particular online identity.
2. Peer discovery—Without a server, it is complex to locate other peers on the network.
3. Network incompatibility—Currently, each popular peer-to-peer application has a set of proprietary protocols that are incompatible with other peer-to-peer networks due to the lack of standards. For example, users on the AIM platform cannot communicate with MSN Instant Messenger users.
4. Platform incompatibility—Software developers must rewrite the low-level core aspects of their peer-to-peer applications for each platform they wish to support. Wireless phones and other mobile devices usually have a limited selection of P2P applications, if any.

JXTA attempts to solve these problems by standardizing the low-level protocols that govern peer-to-peer applications. All JXTA-based P2P applications use identical low-level protocols, hence they are compatible with one another.

Networks built with the JXTA protocols consist of three basic types of entities—peers or peer groups, advertisements and pipes. A peer is any entity that uses JXTA protocols (Fig. 18.14) to communicate with other peers. Each peer need support only some of the protocols, so devices with low processing power and memory can still participate in JXTA networks (albeit with limited functionality). **Peer**

<i>Protocol</i>	<i>Function</i>
Peer discovery	Peers use this protocol to find other entities in the JXTA network by searching for advertisements.
Peer resolver	Peers that help a search process (e.g., send and process requests) implement this protocol.
Peer information	Peers obtain information about other peers via this protocol.
Peer membership	Peers use this protocol to learn about the requirements of groups, how to apply for membership, how to modify their membership and how to quit a group. Authentication and security are implemented through this protocol.
Pipe binding	Peers can connect pipes to one another, via advertisements, through this protocol.
Endpoint routing	Peer routers implement this protocol to provide other routing services to other peers (e.g., tunneling through a firewall).

Figure 18.14 | JXTA low-level protocols.⁶⁷

groups are logical constructs that represent sets of peers. JXTA specifies only two rules regarding peer groups:

1. peers can join or leave groups, and
2. the group administrator, if the group has one, controls access to the group.

Advertisements are XML documents formatted according to JXTA specifications that are used by a peer to advertise itself and notify others of its existence. **Pipes** are virtual communication channels that connect two or more peers for sending and receiving messages among peers. At the simplest level, pipes are one-way communication channels. Two peers communicate by configuring two pipes that "flow" in opposite directions. When a source peer needs to send a message to a destination peer, a pipe is dynamically bound to the two peers. Once the pipe is set up, the source peer can send messages to the destination peer via the pipe.

Examples of P2P applications that uses JXTA are: VistaPortal (www.vistaportal.com/), Momentum by InView Software (www.inviewsoftware.com/news/20030305_momentuml.htm) and ifreestyle by Digital Dream (www.digitaldream.jp/en/).⁶⁸

SelfReview

1. How do AOL Instant Messenger and MSN Instant Messenger bootstrap users onto the network?
2. How do peers communicate in networks built with the JXTA protocol?

Ans: 1) Both AOL Instant Messenger and MSN Instant Messenger use central servers to bootstrap users onto the network. 2) Peers use one-way communication channels called pipes to exchange messages.

18.6 Grid Computing

A few decades ago, the number of computers was relatively small, their computing power was modest and they were expensive and heavily utilized. With the advent of the microprocessor, there are now hundreds of millions of inexpensive, relatively powerful computers in homes and businesses. However, a vast amount of computing power is wasted (e.g., personal computers at home sit idle while people are at work, and work computers sit idle while people are at home). **Grid computing** links potentially vast numbers of computational resources (such as computers, storage devices and scientific instruments) that are distributed over a network to solve complex, massive, computation-intensive problems.

As in other distributed systems, the resources in grid computing are distributed, and users can access them transparently. While clustering typically is localized or centralized, grid computing emphasizes public collaboration. Individuals and research institutes participating in grid computing coordinate resources that are not subject to centralized control. High performance, the goal of grid computing, is achieved by using spare computer power at little or no cost. Grid computing offers better scalability than clusters. For example, the SETI@home project, which is discussed later, runs on half-a-million PCs while an HP XC cluster runs on up to 512 nodes. Grid computing requires sophisticated software to manage distributed computing tasks on such a massive scale.^{69-70,71}

To enable interoperability of heterogeneous distributed resources, a grid-computing system is often divided into five layers. The **application layer**, which is the highest level, contains applications that use the lower layers to access distributed resources. The **collective** layer is responsible for coordinating distributed resources, such as scheduling a task to analyze data received from a scientific device. The **resources layer** enables applications to request and share a resource. The **connectivity layer** carries out reliable and secure network communications between resources. The **fabric layer** accesses physical resources, such as disks.⁷²

The SETI@home (Search for Extraterrestrial Intelligence at home) project (setiathome.ssl.berkeley.edu/) is a popular grid-computing implementation. SETI@home enables individuals to participate in a scientific effort seeking intelligent life elsewhere in the universe. Computers belonging to participants, when not in use, download data representing signals from outer space from the SETI@home server, analyze the data and return the results to the SETI@home server.^{73, 74}

The Globus Alliance (www.globus.org/), based at Argonne National Laboratory, the University of Southern California's Information Sciences Institute, the University of Chicago, the University of Edinburgh and the Swedish Center for Parallel Computers, researches grid-computing resource management and develops software (called the Globus Toolkit) to implement grid computing.⁷⁵ United Devices (www.ud.com) provides grid-computing solutions for businesses.⁷⁶

Self Review

1. What makes grid computing a promising trend?

2. Why is the grid-computing system divided into several layers?

Ans: **1)** Grid computing helps coordinate distributed resources and allows access to potentially massive amounts of computing resources, enabling complex problems to be solved that usually cannot be approached without enormous computing powers. **2)** Distributed resources are typically heterogeneous, so the layered architecture enables interoperability and modularity.

18.7 Java Distributed Computing

Java is used widely to implement distributed systems. We discussed developing distributed systems using Java's RMI in Section 17.3.3, and CORBA in Section 17.3.4. Sun provides an article about CORBA and Java at java.sun.com/j2ee/corba/. The following subsections introduce several additional Java technologies that enable developers to build distributed systems and to share and manage distributed resources.

18.7.1 Java Servlets and JavaServer Pages (JSP)

Java provides a number of built-in networking capabilities that make it easy to develop Internet-based and Web-based applications. This section focuses on both sides of a client-server relationship which is the foundation for the highest-level views of networking in Java, namely **servlets** and **JavaServer Pages (JSP)**. A servlet extends the functionality of a server, most frequently a Web server. Using special Syntax, JSP allows Web-page programmers to create pages that use encapsulated Java functionality and even to write **scriptlets** (Java code embedded in a JSP) of actual Java code directly in the page.⁷⁷

A common implementation of the request-response model is between Web browsers and Web servers that interact via the HTTP protocol. When a user accesses a Web site by means of a browser (the client application), a request is sent to the appropriate Web server (the server application). The server normally responds to the client by sending the appropriate HTML Web page. Servlets enhance the functionality of Web servers to provide capabilities such as secure access to Web sites, interacting with databases on behalf of a client, dynamically generating custom documents to be displayed by browsers and maintaining unique session information for each client.

In some ways, JSPs look like standard HTML or XML documents. In fact, JSPs normally include HTML or XML markup. Such markup is known as fixed-template data or fixed-template text. JSPs are generally used when most of the content sent to the client is static text and markup, and only a small portion of the content is generated dynamically with Java code. Servlets are commonly used when a small portion of the content sent to the client is static text or markup. In fact, some servlets do not produce content. Rather, they perform a task on behalf of the client (such as a database query), then invoke other servlets or JSPs to provide a response. Note that in most cases servlet and JSP technologies are interchangeable. The server that executes a servlet is referred to as the **servlet container** or **servlet**

engine. JSPs are translated into servlets by the servlet container, which then compiles and executes them.⁷⁸

Many developers believe that servlets are appropriate for database-intensive applications that communicate with so-called **thin clients**—applications that require minimal client-side support. With these applications, the server is responsible for database access and clients connect to the server using HTTP. Thus, the presentation-logic code for generating dynamic content can be written once and reside on the server for access by clients, to allow programmers to create efficient thin clients.

Sun Microsystems, through the **Java Community Process**, is responsible for the development of the servlet and JavaServer Pages specifications. The reference implementations of these standards are under development by the **Apache Software Foundation** (www.apache.org) as part of the **Jakarta Project** (jakarta.apache.org).⁷⁹ As stated on the Jakarta Project's home page, "The goal of the Jakarta Project is to provide commercial-quality server solutions based on the Java platform that are developed in an open and cooperative fashion." The servlet and JSP part of the Jakarta Project is called **Tomcat**—this is the official reference implementation of the JSP and servlet standards. The most recent version can be downloaded from the Apache Software Foundation's Web site.

Servlets and JavaServer Pages have become so popular that they are now supported by most major Web servers and application servers, including the Sun ONE Application Server, Microsoft's Internet Information Services (IIS), the Apache HTTP Server, BEA's WebLogic application server, IBM's WebSphere application server, the World Wide Web Consortium's Jigsaw Web server and many more.

Servlet Life Cycle

Architecturally, all servlets implement a common interface, which enables the servlet container to interact with any servlet, regardless of its functionality. A servlet's life cycle begins when the servlet container loads the servlet into memory—usually in response to the first request that the servlet receives. Before the servlet can handle that request, the servlet container initializes the servlet. Then, the servlet can respond to its first request. Typically, each new request results in a new thread of execution (created by the servlet container). The servlet container can also terminate the servlet to release its resources.⁸⁰

JavaServer Pages (JSP)

JavaServer Pages is an extension of servlet technology. JSPs simplify the delivery of dynamic Web content. Web-application programmers can create dynamic content by reusing predefined components and by interacting with components using server-side Java code. JSP programmers can reuse Java objects and create custom tag libraries that encapsulate complex, dynamic functionality. These libraries enable Web-page designers who are not familiar with Java to enhance Web pages with powerful dynamic content and processing capabilities.

There are four key components to JSPs: directives, actions, scriptlets and tag libraries. Directives are messages to the JSP container that enable the programmer

to specify page settings, to include content from other resources and to specify custom tag libraries for use in a JSP. Actions encapsulate functionality in predefined tags that programmers can embed in a JSP. Actions often are performed based on the information sent to the server as part of a particular client request. They also can create Java objects for use in JSP scriptlets. Scriptlets, or scripting elements, enable programmers to insert Java code that interacts with components in a JSP (and possibly other Web-application components) to process requests. The tag-extension mechanism enables programmers to create custom tags. Such tags enable programmers to manipulate JSP content.⁸¹

When a JSP-enabled server receives the first request for a JSP, the JSP container translates that JSP into a Java servlet that handles the current request and future requests to the JSP. If the new servlet compiles properly, the JSP container invokes the servlet to process the request. The JSP may respond directly to the request or may invoke other Web-application components to assist in processing the request. Overall, a JSP's request/response mechanism and life cycle are identical to that of a servlet.

Self Review

1. Suppose a request for a Web page requires the server to generate its content from a database. Which technology discussed in the previous section is most appropriate to accomplish this?
2. Which JSP component enables programmers to insert Java code into JSP?

Ans: 1) Servlets should be used, because JSP is used when a small portion of the content is generated dynamically with Java code. 2) Scriptlets.

18.7.2 Jini

Many network devices, such as printers and Web servers, provide services to network clients. **Jini**—a framework for building reliable and fault-tolerant distributed systems with existing Java technologies—extends this idea of providing services beyond industry computer-based networks and into home-based networks. For example, when you arrive home, your car could use a wireless network to notify your home's lighting service to turn on the lights over the walkway. Each service mentioned here has a well-defined interface. The network-printing service provides an interface that enables applications to print documents. A Web server provides an HTTP interface that enables Web browsers to download documents. Your home's lighting service provides an interface that enables other devices on the network to turn the lights on and off.

To use a Jini service, a Jini client must be able to discover that a service exists and must know the interface for interacting with the service. For example, your car must be able to discover that your home provides a lighting service and must know the service's interface to interact with it. However, the car need not know the implementation of the underlying lighting service.

The Jini **lookup service** is the heart of the Jini architecture (Fig. 18.15). A lookup service maintains information about available Jini services and enables clients to discover and use them. The process of finding the lookup services and obtaining references to them is called **discovery**. A service provider discovers and registers the service interface with one or more lookup services to make itself available to clients. Clients discover the lookup services and request a service they require. The lookup services then send a copy of the service interface to the client. Upon obtaining the service interface, the client communicates with the service provider via RMI, as discussed in Section 17.3.3, using the service interface.⁸²

Recall that Jini services register with lookup services to make the Jini service's functionality available to other members in the network. If all goes well, other members use the service, and the service stays up and running perpetually. In reality, however, services fail for many reasons—network outages can make a service unreachable, a physical device associated with a service (e.g., a printer) might need repairs or the service itself could encounter an unrecoverable error. In these and many other situations, a service could become unavailable, and that service might not be able to unregister itself from lookup services to prevent other clients from attempting to use it.

One goal of Jini technology is to make Jini communities "self-healing" and able to recover from these common problems. Therefore, when a Jini service registers with a lookup service, the registration is not permanent. The registration is leased for a specific amount of time, after which the lookup service revokes the registration (a service provider can also renew the lease anytime before it expires).⁸³ This prevents problematic services from disrupting the entire Jini network. If a Jini

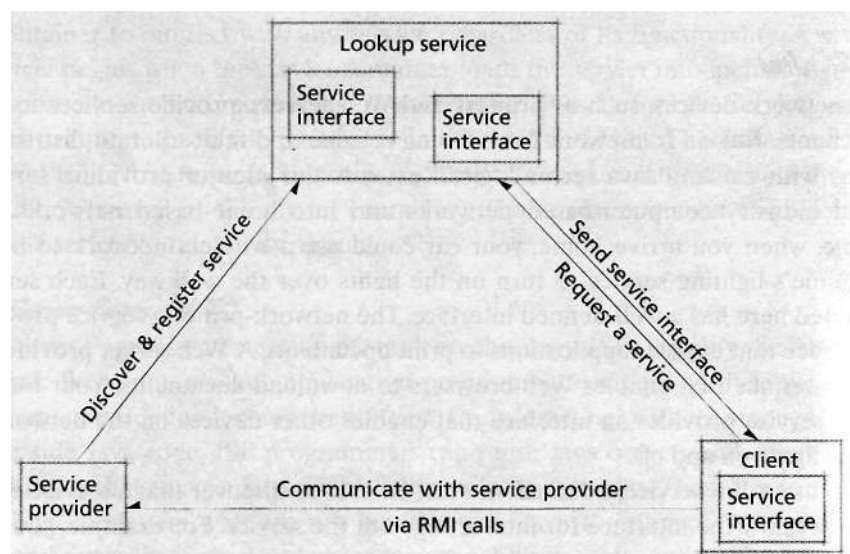


Figure 18.15 | Jini architecture.

service fails, the Jini service's lease eventually expires and lookup services will no longer provide the failed Jini service to clients. The leasing strategy that Jini employs is strict—if a Jini service provider does not renew the lease, the lookup service terminates the registration when the lease expires and the service becomes unavailable to clients.

Self Review

1. What information must a Jini client know in order to use a Jini service?
2. What is the function of the Jini lookup service?

Ans: **1)** The Jini client must know the service interface to use a Jini service. **2)** It acts as a registry for Jini services so that a Jini client can find and use the Jini services that are available.

18.7.3 JavaSpaces

JavaSpaces is a Jini service that implements a simple, high-level architecture for building distributed systems. Objects that take part in distributed systems must be able to communicate with one another and share information. The JavaSpaces service provides distributed, shared storage (and shared memory) for Java objects and enables Java objects to communicate, share objects and coordinate tasks using the storage. Any Java-compatible client can put shared objects into the storage.

A JavaSpaces service provides three fundamental operations—write, take and read, which allow the communications between objects stored in JavaSpaces. The **write operation** adds an object into the JavaSpaces service. When clients want to remove an object, they perform a **take operation**. The take operation removes from the JavaSpaces service an object that satisfies the client's search criteria. The **read operation** is similar to the take operation, but it does not remove the matching object from the JavaSpaces service so that other clients can still find it. Clients can perform read operations concurrently, but only one client can perform the take operation to remove the object from JavaSpaces.⁸⁴

JavaSpaces eases the design and development of distributed systems that share Java objects between the service providers and the clients. A JavaSpaces service has six major properties:⁸⁵

1. A JavaSpaces service is a Jini service.
2. Multiple processes can access a JavaSpaces service concurrently, which helps synchronize actions between distributed applications.
3. An object stored in a JavaSpaces service will remain in the service until its lease expires or until a program removes the object from the JavaSpaces service.
4. A JavaSpaces service locates objects by comparing them to a template. The template specifies the search criteria against which the JavaSpaces service compares each object in the JavaSpaces service. When one or more entries match the template, the JavaSpaces service returns a single matching object.